Letter of Motivation

Application for M.Sc. Computer Science
University of Freiburg, Germany

Dear Members of the Admissions Committee,

I am writing to express my strong interest in pursuing the Master of Science in Computer Science at the University of Freiburg. I completed my Bachelor of Technology in Computer Engineering in 2026 with a CGPA of 8.92/10. Throughout my undergraduate studies, I developed a strong foundation in computer science, along with a growing interest in understanding how computational principles can be applied to design secure, efficient, and reliable software systems. I now wish to strengthen my theoretical knowledge, develop advanced technical skills, and explore complex computing challenges through graduate study.

My academic interests developed particularly through subjects such as Theory of Computation and Compiler Design. Theory of Computation introduced me to formal models, computational complexity, and the fundamental principles governing what computers can accomplish. Compiler Design helped me understand how programming languages are processed through lexical analysis, parsing, syntax analysis, and code generation. These subjects encouraged me to look beyond writing functional programs and examine the underlying structures, algorithms, and design decisions that determine how software behaves. They also strengthened my interest in software engineering, system reliability, and the theoretical foundations of computing.

An important part of my undergraduate education was my bachelor's thesis, titled "Cloud-Native Security and Compliance Data Fabric Platform." The project involved developing a pipeline to detect, process, and route security events before they reached an organisation's Security Information and Event Management (SIEM) system. My responsibilities included testing and service integration, validating event detection and routing logic, and developing automated tests to evaluate system behaviour under different scenarios. Working on this project helped me understand the challenges of building reliable software architectures, particularly when multiple services must communicate accurately and securely. It also showed me how systematic testing, error detection, and software validation contribute to the security and dependability of complex computing environments.

Alongside my academic studies, I gained practical software engineering experience through two internships involving backend development, APIs, databases, and software integration. During my six-month internship, I worked on unit testing, integration testing, and API validation. I also contributed to integrating an automated test suite with TestRail to monitor test results, coverage, and failures. These experiences taught me the importance of structured development practices, clear software architecture, and continuous verification. More importantly, they encouraged me to explore the deeper principles behind software correctness and reliability rather than focusing only on implementation.

The University of Freiburg particularly interests me because of its strong academic environment and its approach to combining fundamental computer science with advanced research and practical applications. I am drawn to the Master's programme because it offers opportunities to develop specialised knowledge while strengthening essential competencies in algorithms, software systems, and computational methods. This balance between theoretical understanding and practical problem-solving closely matches the direction I want to pursue in my academic and professional development.

My primary interests lie in software engineering, computer systems, and information security. Through my bachelor's thesis, I became interested in how distributed software components interact, how vulnerabilities can emerge within complex architectures, and how systematic validation can improve

overall reliability. I would like to explore these challenges further through advanced coursework and research-oriented projects. In particular, I hope to develop a deeper understanding of secure software design, dependable computing systems, and methods for analysing and evaluating software behaviour.

I am also interested in Freiburg's research environment, particularly its work involving algorithms, software systems, and cybersecurity-related topics. The opportunity to engage with researchers and students from different technical backgrounds would help me develop a more systematic approach to investigating complex computational problems. I look forward to participating in academic discussions, working on challenging projects, and eventually undertaking a Master's thesis that allows me to connect theoretical concepts with practical software engineering challenges.

Germany is my preferred destination for postgraduate education because of its strong academic standards, emphasis on independent research, and contributions to computer science and engineering. I particularly appreciate the approach of German universities, where students are encouraged to develop analytical thinking, explore specialised subjects, and apply their knowledge through research and practical projects. Studying in this environment would allow me to broaden my technical perspective while gaining valuable international academic experience.

After completing my Master's degree, I intend to pursue a career in software engineering, cybersecurity, or the development of distributed and interconnected computing systems. My long-term goal is to contribute to the design of reliable and secure software solutions that can address increasingly complex technological requirements. I believe that developing stronger theoretical foundations and research capabilities will help me approach these challenges with greater confidence and technical depth.

Thank you for considering my application.

Yours faithfully,
Nishant Patel

Nishant Patel

Passport: B8408100 **Nationality:** Indian **Date of birth:** 29/03/2005 **Gender:** Male

Phone number: (+91) 8320095397 **Email address:** nishantpatel197322@gmail.com

Home: Flat No E 303, Casario Randesan, 382007 Gandhinagar (India)

ABOUT ME

Computer Engineering student with a strong interest in technology, software development, and emerging engineering fields. Seeking to pursue higher education in Germany to strengthen my technical knowledge, gain international academic exposure, and develop practical skills. I aim to achieve my academic and professional goals while exploring new technologies and opportunities in a global learning environment.

WORK EXPERIENCE

Crest Data – Ahmedabad, India

Software Engineer

[15/12/2025 – 17/07/2026]

• Developed and maintained automated test suites using **Pytest** for the **Databee** application, covering core application workflows and functional scenarios.
• Designed and executed **unit and integration tests** to validate application components, API behavior, and interactions between modules.
• Integrated **TestRail** with the testing workflow to manage test cases, track execution results, and maintain structured test documentation.
• Improved **test coverage** and streamlined the testing workflow through **reusable test cases** and automated validation.

Marwiz Tech Private Limited – Vadodara, India

MERN Developer

[22/05/2025 – 18/07/2025]

• Developed comprehensive **CCTV surveillance platform** with **multi-role access** for administrators, dealers, and clients.
• Built **license management system** for enabling AI-based features such as **crowd, fire,** and **intruder detection.**
• Integrated **RTMP/RTSP-based real-time video streaming** with recorded footage **playback and download** functio nality.
• Implemented **role-based authentication system** with secure **multi-tier access controls** for enhanced security.
• Designed **scalable architecture** to efficiently distribute RTMP/RTSP camera resources across multiple user categories.

EDUCATION AND TRAINING

Preparing for the Higher Education Abroad

[01/08/2026 – Current]

City: Gandhinagar | **Country:** India

IELTS

IDP Education [25/07/2026]

Final grade: Overall Bands: 7.0

Bachelor of Technology in Computer Engineering

Pandit Deendayal Energy University [01/07/2022 – 19/06/2026]

City: Gandhinagar | **Country:** India | **Final grade:** 8.92 CGPA | **Level in EQF:** EQF level 6

Senior Secondary Education 12th

Shree Swaminarayan Gurukul Vidhyalay [01/06/2021 – 30/06/2022]

City: Gandhinagar | **Country:** India | **Final grade:** 75.8 %

Secondary School Education 10th

School of Achiever [01/06/2019 – 30/06/2020]

City: Gandhinagar | **Country:** India | **Final grade:** 92.3 %

PROJECTS

Food Delivery Website | ReactJS, NodeJS, ExpressJS, MongoDB, Stripe • Architected a **REST API** with **Node.js/ Express** supporting multiple vendor accounts with independent menu and order management• Implemented **JWT-based authentication** with role-based access control, and integrated Stripe for secure end-to-end payment processing• Designed and optimized **MongoDB schema**, improving query performance for real-time order tracking and status updates

SKILLS

Web Technologies: HTML5, CSS3 / Java (computer programming) / JavaScript / ReactJS Library / NodeJs, expressJs / C / Git/Github, Docker, Gitlab / Databases : MySQL , MongoDB / Computer Networks and Computer Protocols / use object-oriented programming / Python (computer programming) / Version Control (Git) / PyTest/ PyCharm Framework / FastApi(basic)

LANGUAGE SKILLS

Mother tongue(s): Gujarati

Other language(s): Hindi

English

LISTENING C1 **READING** C1 **WRITING** C1

SPOKEN PRODUCTION C1 **SPOKEN INTERACTION** C1

Levels: A1 and A2: Basic user; B1 and B2: Independent user; C1 and C2: Proficient user

crest(data)

Date: September 24, 2025

Intern Name: Nishant Patel
Ahmedabad

Dear Nishant,

We are pleased to offer an Internship at Crest Data Systems Private Limited ("**Company**" or "**Crest Data Systems**"). We strongly believe that our team is our biggest asset and that our success is directly dependent how we collectively perform together as a team. With that spirit we would like to make the offer below. Please do not hesitate to reach out to us if you have any further questions. We would like to request you to treat all the information in this offer letter as highly confidential and discuss the details only with the HR or your hiring manager.

1. Internship

We are pleased to extend your offer of appointment as an **Intern**. Your designation will be **Software Engineer** at CrestData Systems. Your primary responsibility shall include but not be limited to following:

a) Understand new and programming languages that the Company works with.
b) Complete necessary training provided by the company.
c) Assist project team with Software Development, QA, Automation, CloudOps/SRE, and Support to deliver technology solutions in the field of Data Analytics, Cyber Security, ML/AI, DevOps, and Cloud for the company.
d) Focus on solving customer requirements and perform multiple engineering roles as needed for the project.

The Company shall be at technologies liberty to change the job description when a business necessity for the change arises without any prior intimation.

You agree, as a condition of Internship, that you will execute the Company's '*Non-Disclosure and Confidentiality Undertaking'* Simultaneously with the execution of this letter.

2. Duration and Leave

The internship will start from the date your college allows you to start the Internship to the date the company verifies the Provisional degree certificate from your college. Upon verification of the provisional or final degree certificate by the Company and upon your successful technical evaluation, you will be offered a full- time employment. During the internship period, the Intern is expected to work during regular business working hours and will be subject to the same company working hour policies as a regular employee. Upon verification of the provisional or final degree certificate by the Company, you will be converted to a Full- time employee at the Company.

During the Internship period you are allowed to take up to 1.75 days Paid Time Off (PTO) only for emergencies and study related reasons. Upon completion of internship, PTO leave balance (Positive or Negative) will be converted to cash compensation.

If you are unable to attend work due to illness or injury, you are required upon request to provide a medical certificate from a registered medical practitioner confirming the illness or injury and stating its probable duration. In the case of any illness exceeding the stated probable duration, you will provide a further medical certificate from a registered medical practitioner within [three (3)] days after the expiry of the previous medical certificate.

3. Term of Internship and employment - Stipend and Benefit

Your total monthly stipend will be Rs. 8,000 (Rupees Eight Thousand). Your stipend will be paid monthly and will be subject to deduction of tax at source in accordance with regulations. Your Stipend and benefits may be amended at the sole discretion of the Company. The internship will start from the date the college of Intern allows him/her to start the Internship to the date the Company verifies the Provisional degree certificate from the college of Intern and upon verification of the provisional or final degree certificate by the Company and upon your successful technical evaluation, the Company may offer the employment with the Company with an annual package of **CTC INR 600000/-(Rupees Six lakhs only) Fixed + 100000/- (Rupees One lakh only) Retention Bonus**. The Retention bonus is payable upon

NP

successful completion of one year of employment with the Company ("Due date of Retention Bonus"). The employment shall commence post completion of the internship with the Company. The retention bonus will not be payable in case the employee has resigned/ separated/ serving notice as on the due date of Retention Bonus.

4. Working Location and Norms

Your working location will be CDS House, Nr. Sarkhej-Sanand Circle, SG Road, Makarba, Ahmedabad-382210. As part of your duties, you may be required to travel outside the Company's premises. During your internship with the company, you may also be transferred or deputed to any of the offices or locations of the Company or any of its affiliates or subsidiaries or group companies in India or abroad on the same or similar terms and conditions of Internship.

For Interns working for technical support, DevOps, SRE, CloudOps, Marketing, Sales, Managed Services, or Business Development, it is expected that your regular work hours may be in a shift of 9 hours in the morning / afternoon / evening / night and in rotation across different shifts as required by business. Interns working in the night shift will have their working hours from 10pm to 7 am. These hours may shift slightly depending on the time zone in which the client is based or on the Daylight Savings Time. Interns working for technical support, DevOps, SRE, CloudOps, Managed Services may have rotational weekends and public holidays depending on project requirement.
It is expected that you be available for any customer meetings / engagements as and when necessary.

Regular office working hours are Monday through Friday from 10am to 7pm. However, it is expected that you be available for any customer meetings/ engagements as and when necessary. We currently operate on a five-day work week (Monday to Friday). In the event of travel, accommodation and travel may be provided as per company policy.

5. Termination

a. Your Internship with the Company can be terminated by 15 (fifteen) days' notice (notice period) in writing from either side. However, the Company has exclusive right to accept the resignation and terminate the candidate's Internship any time before the notice period at Company's discretion for any reason whatsoever.
b. However, in the event of gross indiscipline**,** violation of any company policies, willful neglect of duties, breach of trust or any discrepancy or untrue information found in your application, the Company has the discretion to terminate your services with such notice as it deems fit and without any notice or notice pay.
c. Upon the termination of your Internship for any reason by either party, or upon the effective notice of termination of your Internship by either party, the Company shall have the right to relieve you from the performance of all duties of the position upon the continued payment of your salary and Stipend, as then in effect, for the duration of the notice period. During such a notice period, you may be required not to attend for work, at the Company's absolute discretion. However, you will not take up any alternate Internship until the expiration of such notice period.
d. The termination of your Internship howsoever arising shall not affect such of the terms hereof as are expressed to operate or have effect thereafter and shall be without prejudice to any right of action already accrued to the Company in respect of any breach or default by you.
e. If your internship is terminated for any reason:
    i. The Company may set-off amounts you owe the Company against any amounts the Company owes you at the date of termination except for amounts the Company is by law not entitled to set-off.
    ii. You must return all the Company's property to the Company on termination including all written or machine-readable material, software, laptop, mobiles, computers, credit cards, keys and vehicles, etc.; and
    iii. You must not record or retain any confidential information of the Company which is marked as such or which by its nature is deemed as confidential whether so informed by the Company at the time of giving or not, in any form after termination. Upon request of the Company, you will return or destroy or have destroyed all correspondence, specifications, books, documents, market data, cost data, drawings, effects or records etc. or any other material belonging to the Company or relating to its business, including in its electronic form and shall not retain or make copies of these items. You will provide a certificate to the Company that such materials have been destroyed or returned, as the case may be.

NP

6. Expectations

In making this Offer of Internship, we expect that:

a. You devote your full time and attention in carrying out your work activities at the Company and will not engage in or do any other IT business or render any professional service either on a full time or part time basis, you will also not engage yourself directly or indirectly in any other profitable business connected with the dealings of the Company in any way.
b. You will keep your work strictly confidential and not divulge or disclose any information in any form, either during your internship or after, to anyone not employed by the Company. You will also execute and adhere to a Confidentiality Undertaking upon joining the company.
c. You can maintain a state of medical, physical, and mental fitness and ensure annual medical checkups. You may be required to submit a medical fitness certificate.
d. You have accurately represented them in your application and/or resume. You will authorize the Company to verify your educational and Internship records, your conduct, and any other background checks prior to joining the Company or anytime thereafter.

If any of the above expectations are not met, your internship may be terminated forthwith and without any notice period and without any liability to the Company.

7. Protection of Interest

If you conceive or make any new or advanced methods of improving designing / processes / systems or all improvements in relation to the operations of the Company, such developments shall be fully and immediately communicated to the Company and shall be and remain the sole and exclusive right/property of the Company in perpetuity. If required by the Company, you shall execute separate documents or agreements in relation to such matters, conferring such rights on the Company. The salary payable to you hereunder is adequate Stipend for such assignment and hence, you shall not be entitled to any additional payment in this regard.

8. Company Asset Usage Policy including Company issued Laptops:

a. You understand that you are solely responsible for the Company assets whilst in your possession.
b. You shall use the company assets solely for Company-related purpose.
c. The Company's IT and HR team may monitor the Company assets.
d. You shall keep the Company assets in good working order and will notify IT Team in case of any defect or malfunction during use.
e. You shall not install and / or download any unauthorized software and / or applications on Company asset.
f. You shall not allow the Company assets to be used by an unknown or unauthorized person. You shall assume full responsibility for the actions of others while using the laptop.
g. If the laptop is lost, stolen or damaged, the incident must be reported to the IT and HR team and your direct manager immediately.
h. If the lost, stolen or damaged laptop and / or accessories is determined to be caused by negligence or intentional misuse, you shall assume full financial responsibility for repair costs or fair market value of the Company assets.
i. You shall abide by the Company's Information Security and Data Usage policies and sign the Information Security and Data Usage Policy Acknowledgment Form to adhere to the Company's policies upon joining.
j. You are aware that any breach of these policies may render you liable to disciplinary action.

9. Past Records and Documents

If any declaration given or information furnished by you to the Company proves to be false or if you are found to have willfully suppressed any material information, in such cases, you shall be liable to removal from your Internship, without any notice and without any Stipend.

Upon joining the Company, you are required to execute a Non-Compete and Confidentiality and Information Security and Data Usage Policy Acknowledgment Form which along with this offer of Internship, will govern the terms of your Internship with the Company.

NP

As a part of joining process, you will be required to provide verifying certificates/documents regarding your photo identification, PAN card copy, educational qualifications (10th and 12th standard certifications, and Diploma/Bachelors/Master of Engineering degree certification and/or coursework), previous Internship details.

10. Miscellaneous Terms and Conditions

a. You hereby agree and undertake to immediately intimate the Company of any change in the information provided by you.
b. By signing below, you are agreeing that you have read and understood each and every provision of this Letter of Internship and that in consideration of the Company offering you Internship with the Company, you voluntarily and unconditionally agree to abide by its terms and conditions of Internship and any such directions or instructions as may be issued to you by the Company from time to time in discharge of your obligations as an Intern of the Company.
c. If any provision of this letter shall be prohibited by or adjudicated by a court to be unlawful, void, or unenforceable such provision shall to the extent required be severed from this letter and rendered ineffective as far as possible without modifying the remaining provisions of this letter and shall not in any way affect any other provisions or the validity or enforcement of this letter.
d. The Company will have the right to transfer and assign this contract of Internship and all covenants and agreements herein will inure to the benefit of and be enforceable by such successors and assigns. This contract of Internship is personal to you, and you will not be entitled to transfer or assign it in whole or in part.

11. Governing Law and Jurisdiction

This Letter of Internship shall be governed by Indian law. The courts at Ahmedabad shall only have exclusive jurisdiction in the event of any dispute arising between you and the Company in respect of or under this Letter of Internship or in any matter concerning your Internship with the Company. You hereby agree to submit to such exclusive jurisdiction of the courts unconditionally and unequivocally at Ahmedabad as envisaged hereinabove.

Please confirm that the above terms and conditions are acceptable to you and that you accept this Internship by signing this Letter of Internship in duplicate.

We look forward to having you be part of our team and grow. Congratulations and welcome to Crest Data Systems!

Yours Sincerely,

For Crest Data Systems Private Limited

Devanshi Raichura

Name: Devanshi Raichura
Designation: Director HR

Date: September 24, 2025

NP

On signing and acceptance of this Letter of Internship, you have consented to the Company processing, both electronically and manually any appropriate data it may gather during your internship. This may include sensitive personal data, such as medical information, etc.

I agree and accept Internship with the Company based on the terms and conditions mentioned in this Letter of Internship, which have been read and understood by me.

Nishant Patel

_____

Intern Name: Nishant Patel

Date: 24/09/2025

crest(data)

NON-DISCLOSURE AGREEMENT

1. **General**. As an Intern of CREST DATA SYSTEMS PRIVATE LIMITED ("**Crest Data Systems**" or "**Company**"), a company incorporated under the Companies Act 1956 and having its Registered Office at CDS House, Nr. Sarkhej-Sanand Circle, SG Road, Makarba, Ahmedabad-382210 and in consideration of the Stipend now and hereafter paid to me, I will devote my best efforts to furthering the best interests of Crest Data Systems. During my Internship, I will not engage in activity that:

   a. conflicts with Crest Data Systems' business interests, including without limitation, any business activities not contemplated by this agreement,
   b. occupies my attention so as to interfere with the proper and efficient performance of my duties at Crest Data Systems, or
   c. interferes with the independent exercise of my judgment in the best interests of Crest Data Systems.

   As used herein, Crest Data Systems business means the development, QA, Automation, engineering, marketing, technology operations, and support of software and services for business and professional use including operating systems, application programs, Internet related websites or e-commerce solutions as well as books and hardware for the microcomputer and the Internet marketplace.

2. **Recognition of Absolute Ownership**. That I do hereby recognize and admit that Crest Data Systems is the absolute, unrestricted and exclusive owner of the confidential or proprietary technical, financial, marketing, manufacturing, distribution, or other technical or business information or trade secrets of Crest Data Systems, including without limitation, concepts, techniques, processes, methods, systems, designs, clients, cost data, computer programs, formulae, development or experimental work, work in progress, customers and suppliers as well as software for business and professional use including operating systems, application programs, Internet related websites or e- commerce solutions as well as books and hardware for the microcomputer and the Internet marketplace used by me in the course of my Internship with Crest Data Systems.

   I agree that I shall not in any manner whatsoever, represent and/or claim that I have any interest by way of ownership, assignment or otherwise in the same.

   In this agreement, all confidential and/or proprietary information belonging to and/or in possession of Crest Data Systems, which is received, accessed, and/or used by me during the course of my Internship with Crest Data Systems, shall include without limitation, such information received from Crest Data Systems, its customers and/or any entity in which Crest Data Systems holds or controls more than 50% of the equity stock thereof and/or is entitled to vote for the election of directors.

3. **Non-Disclosure**. At all times, during my internship and thereafter, I will not disclose to anyone outside Crest Data Systems nor use for any purpose other than my work for Crest Data Systems:

   a. any confidential or proprietary technical, financial, marketing, manufacturing, distribution, personal data that you may have access to as part of your project with Crest clients or other technical or business information or trade secrets of Crest Data Systems, including without limitation, concepts, techniques, processes, methods, systems, designs, circuits, cost data, computer, programs, formulae, development or experimental work, work-in-progress, customers, and suppliers,
   b. any information Crest Data Systems has received from others which Crest Data Systems is obligated to treat as confidential or proprietary or
   c. any confidential or proprietary information which is circulated within Crest Data Systems via its internal electronic mail system, intranet or otherwise.

   I will also not disclose any confidential or proprietary information to anyone inside Crest Data Systems except on a "need-to-know" basis. If I have any questions as to what comprises such confidential or proprietary information or trade secrets, as to whom, if anyone inside Crest Data Systems, it may be disclosed, I will consult with my manager and Human Resources (HR) at Crest Data Systems.

4. **Assignment of Inventions**. I hereby assign exclusively to Crest Data Systems all my right, title, and interest in and to any all inventions, discoveries, designs, developments, improvements, copyrightable material, and trade secrets (collectively herein "**Inventions**") that I solely or jointly may conceive, write, encode, develop, test, or

NP

reduce to practice during the period of time I am in the Internship of Crest Data Systems. I will make prompt and full disclosure to Crest Data Systems of any inventions, and if for any reason the assignment pursuant to this clause is not effective, will hold all such inventions in trust for the sole benefit of Crest Data Systems.

I hereby waive and quit claim to Crest Data Systems, all claims of any nature whatsoever that I now or hereafter may have for infringement of any patent resulting from any patent applications for any inventions so assigned to Crest Data Systems.

My obligation to assign shall not apply to any invention about which I can prove that:

a. it was developed entirely on my own time; and
b. no equipment, supplies, facilities, services or trade secret of Crest Data Systems was used in its development; and
c. it does not relate
    i. directly to the business of Crest Data Systems or
    ii. to the actual or demonstrably anticipated research or development of Crest Data Systems; and
d. it does not result from any work performed by me for Crest Data Systems.

5. **Excluded and Licensed Inventions**. I have attached hereto, a list describing all inventions – belonging to me and made by me prior to my Internship with Crest Data Systems that I wish to have excluded from this agreement. If not, such list is attached, I represent that there are no such inventions. If in the course of my Internship at Crest Data Systems, I use in or incorporate into a Crest Data Systems product, program, process or machine, and invention owned by me or which I have an interest, Crest Data Systems is hereby granted and shall have an exclusive royalty-free, irrevocable, worldwide license to make, have made, use and sell that invention without restriction as to the extent of my ownership or interest.

6. **Application for Copyright and Patents**. I will execute any proper oath or verify any proper document in connection with carrying out the terms of this agreement. If, because of my mental or physical incapacity or for any other reason whatsoever, Crest Data Systems is unable to secure my signature to apply for or to pursue any application for any Indian or foreign patent or copyright covering inventions assigned to Crest Data Systems as stated above, I hereby irrevocably designate and appoint Crest Data Systems and its duly authorized officers and agents as my agent and attorney in fact, to act for me and in my behalf and stead, to execute and file any prosecution and issuance of Indian and foreign patents and copyrights thereon with the same legal force and effect as if executed by me,. I will testify at Crest Data Systems request and expense in any interference, litigation or other legal proceeding that may arise during or after my internship.

7. **Third Party Information**. I recognize that Crest Data Systems has received and will receive confidential or proprietary information from its customers as well as third parties subject to a duty on Crest Data Systems part to maintain the confidentiality of such information and to use it only for certain limited purpose. During the term of my internship and thereafter, I will not disclose such confidential or proprietary information to anyone except as necessary in carrying out my work in Crest Data Systems and consistent with Crest Data Systems agreement with such customers or third party. I will not use such information for the benefit of anyone other than Crest Data Systems or such third party, or in any manner inconsistent with any agreement between Crest Data Systems and such third party of which I am made aware.

8. **Prior Intern Information**. During my Internship at Crest Data Systems, I will not use improperly or disclose any confidential or proprietary information or trade secrets of my former or current employers, principals, partners, co-ventures, clients, customers or suppliers of the vendors or customers of such persons or entities or their vendor or customers and I will not bring onto the premises of Crest Data Systems, any unpublished document or any property belonging to any such persons or entities or their vendors or customers unless such persons or entities have given their consent. I will not violate any non-disclosure or proprietary rights agreement I might have signed in connection with any such person or entity.

9. **Presumption of Breach**. In the event of the possession, access and or use of the confidential or proprietary technical, financial, marketing, manufacturing, distribution or other technical or business information or trade secrets of Crest Data Systems, including without limitation, concepts, technique's processes, methods, system's, designs, clients, cost data, computer programs, formulae, development or experimental work, work-in-progress, customers and suppliers as well as software for business and professional use, application programs, internet

NP

websites, e-commerce solutions, books, hardware and information for the microcomputer and internet marketplace by any other third party with whom I may have a nexus, it shall be presumed, unless proved to the contrary, that such information has so come to the possession of the third party on account of breach of this agreement by me.

10. **Term of Internship**. I acknowledge that my internship will be of indefinite duration and that either Crest Data Systems or I will be free to terminate this Internship relationship at will and at any time with or without cause and in accordance with the Internship Agreement signed by me with Crest Data Systems on the day of joining. I also acknowledge that any representations to the contrary are unauthorized and void, unless contained in the said Internship agreement signed by an officer of Crest Data Systems.

11. **Return of Materials**. At the time I leave the Internship of Crest Data Systems, I will return to Crest Data Systems all papers, drawings, notes, memoranda, manuals, specifications, designs, devices, documents, diskettes, CDs, DVDs, Tapes, DAT Drives, and any other material on any media containing or disclosing any confidential or proprietary technical or business information. I will also return any keys, pass cards, ID cards or other property belonging to Crest Data Systems.

12. **Non-Solicitation**. During the Internship at Crest Data Systems and for a period of 2 years from the termination of my Internship, I will not induce or attempt to influence directly or indirectly, any intern at Crest Data Systems to terminate his Internship with Crest Data Systems or to work for me or any other person or entity.

    a. Non-solicitation of clients. I agree for a period of 2 Years from the date of relieving of my Internship with Crest Data Systems to not directly or indirectly solicit competitive business from any client or customer of the organization that was contacted, solicited, or served by me or about which I received confidential information while I was interning by Crest Data Systems, nor for the same period of time, will I perform services or accept any business, competitive with that of Crest Data Systems, directly or indirectly from any of the clients or customers of Crest Data Systems, which involves me performing similar functions or acting in a similar capacity as when Interned with Crest Data Systems.
    b. Internship with clients. I agree for a period of 2 Years from the date of relieving of my Internship with Crest Data Systems to not perform services, Internship, or accept any business, that competes with that of Crest Data Systems directly or indirectly from any of the clients or customers of Crest Data Systems, which involves me performing similar functions or acting in a similar capacity as when Interned with Crest Data Systems.
    c. Non-solicitation of other employees. I agree so long as Interned by Crest Data Systems and for a period of 2 Years after leaving for any reason whatsoever, not to directly or indirectly recruit, solicit, or otherwise induce or attempt to induce any employee of Crest Data Systems to terminate his or her Internship with the Company or otherwise to act contrary to the interests of Crest Data Systems.
    d. Liquidated Damages. I, unconditionally, without any protest and demur, agree to pay sum of Rs. 30,00,000/- (Rupees Thirty lacs Only) as a liquidated damage, to Crest Data Systems, in case of breach of the clause no. 12a, 12b, or 12c of this agreement on my part. The figure of liquidated damages is arrived at mutually and shall not have any bearing whether actual damages is caused to the tune of Rs. 30,00,000/- or not. I further agree that this clause shall not preclude Crest Data Systems from recovering the further actual damages, if the same exceeds an amount of Rs. 30,00,000/-. The figure arrived by the Crest Data Systems with respect to the damages will be final and binding and I shall have no right to dispute the same.
    e. I further agree that crest data system has absolute and unconditional right to revise the figure of the liquidated damages. It is agreed between the parties that Crest Data Systems will revise the liquidated damages every year and the same shall be posted in Company's Employee handbook. The revised figure so arrived by the Crest Data Systems is binding to me. I further unconditionally agree not to protest and contest the said revised figure by recourse to Law or otherwise.

13. **Personal Property**. I agree that Crest Data Systems will not be responsible for loss, disappearance, or damage to personal property on Crest Data Systems premises, or if applicable, on residential premises subsidized by Crest Data Systems (including apartments or temporary housing). I hereby release, discharge and hold Crest Data Systems harmless from any and all claims relating to loss of, disappearance, or damage to such personal property.

14. **Equitable Relief**. I acknowledge that any violation by me under this agreement, and/or any obligation of like nature, will cause irreparable injury to Crest Data Systems, and Crest Data Systems shall be entitled to

NP

extraordinary relief in any court in India, including, but not limited to, temporary restraining orders, preliminary injunctions, and permanent injunctions, without the necessity of posting bond or security.

15. **Attorney Fees**. If court proceedings are required to enforce any provision of this agreement, the prevailing party shall be entitled to an award of reasonable and necessary expenses of litigation, including reasonable attorney fees.

16. **Entire Agreement**. I agree that this agreement shall be governed for all purposes by the laws of India. If any provision of this agreement shall be declared excessively broad, it shall be construed to afford Crest Data Systems the maximum protection permissible by law. If any provision of this agreement is void or is so declared, such provision shall be severed from this agreement, which shall otherwise remain in full force and effect. This agreement sets forth the entire agreement of the parties as to the subject matter hereof and any representations, promises, or conditions in connection therewith not in writing and signed by both parties shall not be binding upon either party, the terms and conditions of this agreement shall survive termination of my Internship.

17. The Courts of Ahmedabad alone shall have exclusive jurisdiction in case of dispute with respect to interpretation and implementation of the clauses of this agreement.

HAVING READ AND FULLY UNDERSTOOD THIS AGREEMENT, I HAVE SIGNED MY NAME DATED BELOW:

I agree and accept internship with the company based on the terms and conditions mentioned in this letter of internship, which have been read and understood by me.

For Crest Data Systems Private Limited

Devanshi Raichura

_____

Name: Devanshi Raichura
Designation: Director HR

Date: September 24, 2025

Nishant Patel

_____

Intern Name: Nishant Patel

(I have singed this agreement voluntarily, without any coercion and undue pressure)

Date: 24/09/2025

Completed Document Audit Report

Completed with SignWell.com

Title: Internship Offer Letter - Nishant Patel

Document ID: a64751cc-237d-4436-8777-94d452c122a6

Time Zone: (GMT+00:00) Coordinated Universal Time

Files

Internship Offer Letter -.docx - 9 pages	Sep 24, 2025 16:26:35 UTC

Activity

Stuti Shah IP: 163.116.205.113	created the document (stuti.shah@crestdata.ai)	Sep 24, 2025 16:26:36 UTC
Stuti Shah	sent the document to devanshi.raichura@crestdata.ai	Sep 24, 2025 16:27:41 UTC
Devanshi Raichura IP: 163.116.213.80	Successfully entered passcode (devanshi.raichura@crestdata.ai)	Sep 24, 2025 17:37:32 UTC
Devanshi Raichura IP: 163.116.213.80	first viewed document (devanshi.raichura@crestdata.ai)	Sep 24, 2025 17:37:33 UTC
Devanshi Raichura IP: 163.116.213.80	signed the document (devanshi.raichura@crestdata.ai)	Sep 24, 2025 17:37:41 UTC
Stuti Shah	sent the document to nishantpatel197322@gmail.com	Sep 24, 2025 17:37:41 UTC
Nishant Patel IP: 2402:a00:10a:48bd:34dc:4865:3ff5:c4b7	Successfully entered passcode (nishantpatel197322@gmail.com)	Sep 24, 2025 17:40:20 UTC
Nishant Patel IP: 2402:a00:10a:48bd:34dc:4865:3ff5:c4b7	first viewed document (nishantpatel197322@gmail.com)	Sep 24, 2025 17:40:21 UTC
Nishant Patel IP: 2402:a00:10a:48bd:34dc:4865:3ff5:c4b7	signed the document (nishantpatel197322@gmail.com)	Sep 24, 2025 18:02:08 UTC

MARWIZ

Date: 03/06/2025

Dear Nishant Patel,

Subject: Internship training for Trainee Developer

Welcome to MARWIZ Tech Pvt Ltd...

We hereby offer you internship for the post of Jr. MERN Stack developer. During your internship period you are supposed to work on all MERN Stack development tasks. Apart from your regular training description, you have to get involved in work areas like Web development, SEO, Designing & Graphics and other work areas at MARWIZ.

Your internship will be for period of 2 Months and your stipend amount will be 5,000/- per month.

After completion of your internship programme with us we will confirm your placement at

MARWIZ. Subject to your placement with us.

Your date of joining is 22/05/2025.

Expecting Creative mindset in upcoming days, Wishing you good luck for future endeavours!!

Yours Faithfully,

For MARWIZ Tech Pvt Ltd

MARWIZ

(R. R. Shah)

C.E.O

ADDRESS
Darpan Apartment, 305, next to Express Hotel, Alkapuri, Vadodara, Gujarat 390005

PHONE
+91 6358811021
+91 9824091021

EMAIL
rupesh@marwiz.in
harshal@marwiz.in

WEBSITE
www.marwiztech.com
www.marwiz.in

MARWIZ

Date: 18 Aug2025

Certificate of Internship Completion

This is to certify that **Mr. Patel Nishant** student of Pandit Deendayal Energy University, Gandhinagar has successfully completed internship training for the period of **2 Months** at our organization.

He has successfully completed internship as **Full Stack Intern**.

During his internship program with us he was found punctual, hardworking & inquisitive. He has worked whole heartedly and intelligently to complete the project in his defined time frame. We wish him all the best for his future projects!!

Regards,



Marwiz Tech Pvt. Ltd.

(R. R. Shah)

UGC Recognized

PANDIT DEENDAYAL ENERGY UNIVERSITY

Raisan, Gandhinagar – 382 426, Gujarat - INDIA. Website : www.pdeu.ac.in

NAAC Accredited 'A++' Grade (CGPA 3.52 out of 4.00)

Date : 17-06-2026

BONAFIDE CERTIFICATE

This is to certify that **Mr. Patel Nishant Jayeshkumar, Roll No. 22BCP027,** is a bonafide student of the School of Technology, a constituent institute of Pandit Deendayal Energy University. He is currently enrolled in four-year B. Tech programme, (Specialization - Computer Engineering), for the academic batch of 2022 - 2026.

This document is issued at the request of the student for the purpose of 'Study abroad' and affects no legal / guarantee or obligation.

Please feel free to contact the School of Technology, Pandit Deendayal Energy University at 079-23275405/06/12 if you have any questions or concerns regarding this information.

Sincerely,

Director
School of Technology
Pandit Deendayal Energy University

UGC Recognized

PANDIT DEENDAYAL ENERGY UNIVERSITY

Raisan, Gandhinagar – 382 426, Gujarat - INDIA. Website : www.pdeu.ac.in

NAAC Accredited 'A++' Grade (CGPA 3.52 out of 4.00)

To Whom It May Concern

This is to certify that **Mr. PATEL NISHANT JAYESHKUMAR**, Roll No. **22BCP027**, has successfully passed all semester examinations **without any backlogs** for the **Academic Batch 2022-2026** and has **completed the B.Tech.(Computer Engineering)** degree from this University.

Date : 17-06-2026

Controller of Examinations

Cloud-Native Security and Compliance Data Fabric Platform

Comprehensive Project Report

Submitted in Partial Fulfillment of the Requirements for the Degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE & ENGINEERING

By

Nishant Patel

22BCP027

Under the Guidance of

Dr. Ashish Patel



Department of Computer Science & Engineering,
School of Technology, Pandit Deendayal Energy University,
Gandhinagar 382 426

May 2026

Acknowledgement

Completing this project was never a solo endeavour, even if the keyboard strokes were mine alone. Over the past several months, I have had the privilege of working on a problem that sits at the intersection of cloud computing, data engineering, and cybersecurity three fields that are individually demanding and collectively humbling. I could not have navigated this without the people around me.

To start with, I wish to thank my project guide, **Dr. Ashish Patel** for her valuable guidance and constructive feedback in completing the project. He played an instrumental role in helping me understand the concepts related to cybersecurity operation and cloud technology.

Next, I would like to thank **Mr. Parth Pedhadiya** and Crest data as well for allowing me the chance to be associated with such a great project which was conducted in a professional cybersecurity environment.

I also wish to thank Pandit Deendayal Energy University Department of Computer Science & Engineering for creating an ideal learning environment for me.

Last but not least, I want to thank my family and friends for constantly encouraging me throughout the course of my internship and the project period.

Nishant Patel

Abstract

The growth of security telemetry in enterprise environments advanced is such that traditional ways of managing and dealing with data start to break down. Most organizations send all their logs, events and alerts into a Security Incident and Event Management (SIEM) system to serve as the brain of their security operations. Nevertheless, SIEM platforms come with licensing models that depend on the volume of data ingested. With the rise of cloud adoption, container workloads, identity systems and an ever-expanding range of endpoints, the volume of generated telemetry has exploded. As a result, running these systems has become an excessive operational cost.

It's not just a money matter. When all raw telemetry, regardless of its security significance, enters the SIEM at the same level of priority, analysts are flooded with data that obscure the signals they seek. The "haystack problem" describes the phenomenon where important alerts become increasingly difficult to find among an opaque mass of low-signal events. The measurable impacts of this include alert fatigue, missed detections, and delayed incident response.

This project outlines a cloud-native data fabric solution as an intelligent middleware layer that lies between the source security data and the target SIEM or SOAR solutions for further analysis. Ingesting data from various sources including endpoint, network, and cloud provider telemetry, identity and access management systems, and vulnerability scanners, the platform goes through the process of normalizing the schema, enriching events with metadata, and correlating related events together. Events with no security significance get archived straight to the data lake or discarded entirely depending on the configuration; events with enrichment and correlations that make their significance clear are sent to the SIEM. As a consequence, fewer but more relevant data pass through the expensive system while still retaining comprehensive security coverage.

In addition to being a simple data fabric that routes the input security events to the proper destination, the proposed platform features a detection engine capable of running detection

rules specified by security experts in a format agnostic to a specific vendor, such as Sigma rules. This way, security detections can be performed in the data lake, which allows security professionals to refine their detection rules and deploy results however they see fit.

The implementation was proven via the use of a component test harness written in Python and using pytest, conducted within Kubernetes pods, across five different categories of tests, namely, asset hygiene, detection chaining, rules and Sigma detection, security hygiene, and risk flow analysis. The experimental findings reveal a decrease of 70-73% in the amount of data transferred to the SIEM system, with more than 91% accuracy in detecting malicious entities in validated test data. These findings imply that the suggested solution is a feasible approach to the problem of SIEM cost reduction without sacrificing existing tools.

INDEX

Chapter No.	Section No.	Title	Page No.
		Acknowledgement	i
		Abstract	ii
		Index	iv
		List of Figures	vi
		List of Tables	vi
		Nomenclature	viii
1		Introduction	1
	1.1	Background	1
	1.2	Motivation	3
	1.3	Objective	4
	1.4	Problem Statement	5
	1.5	Approach	6
	1.6	Scope of the Project	7
2		Literature Review	8
	2.1	Previous Approaches to Solve the Problem	8
3		Hardware Design	14
	3.1	Infrastructure Overview	14
	3.2	Cloud Compute Configuration	15
	3.3	Kubernetes Cluster Architecture	16
	3.4	Storage Infrastructure	18
	3.5	Networking and Security Infrastructure	20
	3.6	Containerization Strategy	21
4		Software Design	22

Chapter No.	Section No.	Title	Page No.
	4.1	Overall System Architecture	22
	4.2	Data Ingestion Layer	24
	4.3	Correlation and Detection Engine	25
5		Results and Discussion	27
	5.1	Experimental Setup	27
	5.2	Test Methodology	27
	5.3	Component Test Results	29
	5.4	Performance Benchmarks	29
	5.5	SIEM Data Volume Reduction Analysis	30
	5.6	Comparison with Existing Approaches	31
	5.7	Discussion	32
6		Conclusions and Future Scope	34
	6.1	Summary of Contributions	34
	6.2	Conclusions	35
	6.3	Limitations	36
	6.4	Future Scope	37
		References	39
		Appendix	41

LIST OF FIGURES

Figure No.	Title	Page No.
3.1	Kubernetes Cluster Node Layout And Pod Scheduling	15
3.2	Storage Layer Architecture Bucket Hierarchy And Data Tiers	19
4.1	Overall System Architecture Of The Data Fabric Platform	24
4.2	End-To-End Data Flow Through The Processing Pipeline	25
A.1	Dashboard Overview	41
A.2	Datalake Configuration	41
A.3	Different Feed Support	42

LIST OF TABLES

Table No.	Title	Page No.
3.1	Kubernetes Worker Node Resource Specifications	15
4.1	Technology Stack Summary for Platform Implementation	23
5.1	Summary Of Component Test Results Across All Test Categories	28
5.2	SIEM Data Volume Comparison Baseline Versus Post-Deployment	31

NOMENCLATURE

Symbol / Abbreviation	Full Form
SIEM	Security Information and Event Management
SOAR	Security Orchestration, Automation and Response
API	Application Programming Interface
REST	Representational State Transfer
ETL	Extract, Transform, Load
IaC	Infrastructure as Code
YAML	Yet Another Markup Language
JSON	JavaScript Object Notation
SDK	Software Development Kit
CI/CD	Continuous Integration / Continuous Deployment
K8s	Kubernetes
AWS	Amazon Web Services
S3	Simple Storage Service
IOC	Indicator of Compromise
EDR	Endpoint Detection and Response
NDR	Network Detection and Response
XDR	Extended Detection and Response
TTP	Tactics, Techniques, and Procedures
MITRE	MITRE ATT&CK Framework
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
RBAC	Role-Based Access Control
TLS	Transport Layer Security
VPC	Virtual Private Cloud
IAM	Identity and Access Management
EPS	Events Per Second
OSINT	Open Source Intelligence
ML	Machine Learning
OCSF	Open Cybersecurity Schema Framework
ECS	Elastic Common Schema
SOC	Security Operations Centre
UEBA	User and Entity Behaviour Analytics

Symbol / Abbreviation	Full Form
TTL	Time To Live
LRU	Least Recently Used
GB	Gigabyte

Chapter 1

INTRODUCTION

1.1 Background

The cybersecurity landscape has undergone a structural change over the past decade. Organizations used to operate within a relatively bounded perimeter – a corporate network, a data center, a fixed set of endpoints – but now operate across sprawling hybrid environments including on-premises infrastructure, multiple public cloud providers, containerized workloads, remote devices, third-party SaaS applications and an ever-expanding list of identity systems. Each of these surfaces generates security telemetry in ever-increasing volumes. Endpoints generate process execution logs and file access records. Cloud platforms generate detailed audit trails of every API call and configuration change. Identity providers document authentication events, access grants and privilege escalations. Network monitoring tools capture flow data and packet metadata. Organizations used to manage a fairly defined perimeter—a corporate network, a data centre, a static set of endpoints—now they run across sprawling hybrid environments made up of on-premises infrastructure, multiple public cloud providers, containerized workloads, remote devices, third-party SaaS applications, and an ever-growing list of identity systems. Each of these surfaces generates security telemetry at ever-increasing volumes. Endpoints generate process execution logs and file access logs. Cloud platforms provide detailed audit trails of every API call and configuration change. Authentication events, grants of access, and privilege escalations are logged by Identity providers. Network monitoring tools collect flow data and packet metadata.

SIEM vendors typically charge per gigabyte or per event ingested. As volumes of telemetry have grown, so have licensing costs, sometimes in a non-linear fashion. Organizations have cited SIEM licensing as one of their top three security expenditure line items [2], a situation forcing difficult trade-offs between data retention periods, query performance, and breadth of monitoring coverage. Some teams respond by selectively suppressing telemetry at the source, by not collecting certain event types to remain within budget. This is not cost

management; it is a conscious reduction in security coverage driven by financial constraint. As telemetry volume has increased, so too have the licensing costs associated with it, sometimes in a non-linear fashion. [2] Organizations report SIEM licensing as one of their top three security spending line items, making for difficult trade-offs between data retention periods, query performance, and breadth of monitoring coverage. Some teams do this by selectively suppressing telemetry at the source, choosing not to collect certain event types in order to keep within budget. This is not cost management, this is a deliberate scaling back of security coverage in the face of financial constraint."

Beyond the direct cost problem, there is a more insidious degradation of operations that does not show up on any invoice. When every collected log record is fed into the SIEM with equal priority, regardless of whether it is a routine scheduled task or an attacker's lateral movement attempt, the signal-to-noise ratio suffers. An analyst combing through the alert queue sees a mixture of genuine threats, benign anomalies, poorly tuned detection rules, and routine system activity that was never worth alerting on. teams have found analyst burnout rates well above those of comparable technical roles, with alert fatigue cited consistently as a contributing factor [3]. The data volume problem is, in this sense, also a human problem one with direct consequences for detection quality and analyst retention. Pushing every collected log record into the SIEM with the same priority—whether it is a routine scheduled task or an attacker's attempt at lateral movement—degrades the signal to noise ratio. The analyst reviewing the alert queue will see a mix of real threats, benign anomalies, poorly tuned detection rules, and normal system activity that should never have triggered an alert. Teams have found that analysts burn out at a rate far higher than other comparable technical roles, with "alert fatigue" consistently cited as a contributing factor [3]. In this sense, the problem of data volume is also a human problem with direct consequences for detection quality and analyst retention.

1.2 Motivation

The reason behind undertaking this project was based on the observation made throughout the duration of the project. In particular, the observation involved interactions with the enterprise cybersecurity software firm that deals with creating software solutions for handling large-scale security data infrastructure. Empirical observations of how security operations teams operate, how they work with their data, and the areas of conflict played an even greater role in determining the direction of this project than the results of any literature reviews might have done.

The first observation had to do with money. In organizations dealing with several hundred gigabytes of security data per day, license fees for the SIEM solution take up quite a chunk of the entire budget of the security program. Sometimes, it can account for more than one third of all expenditures on security tools. The need to save on this expense is both clear and pressing. At the same time, the typical response is rather negative, involving either data suppression, which undermines the coverage, or spending the money under the burden of the restriction imposed by it. Neither of these approaches qualifies as a proper solution. What the companies need is a platform that could help them lower SIEM-related expenses by filtering out unnecessary data using smart routing and archiving the rest efficiently and cost-effectively.

The second observation related to the quality of detection. While not all enterprises struggle with expensive SIEM license, the sheer quantity of the low-context events in the system creates its own problems. Indeed, rules built based on the raw, unprocessed events produce more false positives compared to rules based on the enriched data. This is because raw data does not distinguish between connections from malicious addresses and those coming from home computers of the employees. Thus, a lot of analysts spend their time sorting through the false alarms rather than investigating threats properly.

The third observation concerned detection portability. Security teams that invest significant time in building detection rule libraries in one SIEM's proprietary query language cannot transfer that investment if they need to change SIEM platforms. The existence of the Sigma

open-source detection format a vendor-agnostic YAML-based rule language that can be compiled to multiple SIEM backends pointed toward a better approach: executing detection rules at a platform layer that is independent of any downstream SIEM, forwarding only the resulting findings. This approach simultaneously addresses detection portability and reduces the data burden on the SIEM.

Together, these three observations defined the design space clearly enough to attempt a concrete implementation.

1.3 Objective

The project sets out to achieve five objectives that relate to one another.

To begin with, concerning the architectural considerations, the goal is to design a modular, cloud-ready architecture that allows collecting security telemetry from different sources – push sources, pull sources, and files, without requiring modifications to the sources themselves or forcing conformity to a single data format. Such an ingestion layer should provide sufficient flexibility to incorporate new source types by configuration alone, without writing new code.

As far as the data quality aspect goes, the goal is to introduce normalization and enrichment capabilities, where heterogeneous schemas used by sources are transformed into an internal data model while additional metadata is applied to each event. Each enriched and normalized event is expected to offer greater analytical value relative to the volume compared to its unmodified counterpart.

With regard to detection independence, the project seeks to develop a correlation and detection engine that supports running custom detection rules, as well as rules written in the Sigma syntax, within the data fabric layer, without using SIEM-specific query languages. At the same time, the engine must support detection based on single events, as well as pattern detection via a detection chaining capability.

The goal relating to measurable cost reduction implies proving that it is possible to forward less than 40% of all the collected data to the SIEM, compared to the original amount, without reducing detection completeness. It was concluded that this would make a statistically significant reduction in costs.

Last but not least, the goal related to accessibility concerns designing rule detections, enrichment mappings, and data routing policy expressions in such a way that they could be understood by security professionals lacking a background in software engineering.

1.4 Problem Statement

In enterprise settings, where large volumes of security telemetry are produced, there is a need to solve two related problems. The first one relates to the high expenses of sending all the collected telemetry into a SIEM tool, which depends on data volume, thus creating cost-related pressure leading enterprises to narrow down telemetry collection for cost reduction purposes. The second problem concerns the absence of an intelligent layer pre-processing data collected from various sources before forwarding the data to SIEM, which entails the arrival of unenriched and uncorrelated events to the analytics engine.

Current solutions help to address partially the issues mentioned above. For instance, log aggregators and forwarders may help to optimize SIEM data volume using deduplication and simple filtering techniques; however, they provide only partial savings (around 20-35%) because routing decisions are not based on enrichment context [17]. SIEM-side filtering solves the problem moving its part to another step – events should still get inside SIEM boundary to be filtered out [4]. Security data lakes help to save costs on storage but have not historically been equipped with real-time enrichment, correlation, and seamless SIEM integration capabilities [15]. Sigma helps in detecting portability through rules creation but not execution.

Thus, no existing solution covers both enrichment-aware intelligent routing, in-fabric execution, and seamless integration aspects at once.

1.5 Approach

The current project is dedicated to the development and deployment of a cloud-native data fabric platform which will act as an intelligent middleware layer between the sources of security telemetry and subsequent SIEM/SOAR systems. The processing pipeline in such a layer consists of four stages: normalization, enrichment, correlation and detection, and routing.

Normalization is responsible for converting incoming security telemetry events from vendor-specific schemas into a unified internal representation. As such, this step allows further stages to proceed without having to include any specific logic regarding different telemetry sources. Enrichment stage adds additional context-based metadata to every normalized event using asset, identity, and threat intelligence registries. Correlation uses detection rules defined by users to detect security incidents both in a single event and among multiple events using the detection chaining procedure. Finally, the routing uses results from the previous step and determines whether the event needs to be passed to SIEM as an enriched event, as a detection finding, or stored to the data lake.

This project employs a set of independent containers deployed in Kubernetes to implement the proposed middleware. These microservices interact with one another using an internal message queue system that allows decoupling of pipeline stages and providing some fault tolerance against any possible load imbalance in the process. Additionally, a test harness based on Python and pytest frameworks running in a Kubernetes pod ensures consistent testing of all components of the project.

Development was done iteratively, i.e., first, we built the component of interest, tested it, profiled it, and made necessary changes before building the next component. Though this slowed our progress somewhat compared to concurrent development, it helped us debug problems early on and delivered better results in the end.

1.6 Scope of the Project

This project covers the full design and implementation of the data fabric platform's core processing pipeline the ingestion layer, normalization engine, enrichment service, correlation and detection engine, data lake integration, and SIEM/SOAR output adapter along with the component testing framework used to validate platform behaviour.

The deployment model is Kubernetes-based and fully containerized with Docker. The cloud storage layer uses S3-compatible object storage, with data stored in Apache Parquet columnar format. The detection rule format is compatible with the Sigma open-source specification.

The scope explicitly excludes a full analyst dashboard or SIEM-equivalent visualization interface the platform is a backend data processing and routing layer, not a SIEM replacement. It also excludes the development of source-specific data connectors for every possible telemetry type; instead, the platform exposes a generic ingestion API that any source-specific connector can target. Building individual connectors for specific third-party security products would require separate projects for each.

Chapter 2

LITERATURE REVIEW

2.1 Previous Approaches to Solve the Problem

Understanding the landscape of prior work is essential for accurately situating the contribution of this project. The problem of efficiently managing large volumes of security telemetry has attracted research and engineering effort from multiple angles SIEM architecture and cost management, pre-SIEM filtering and aggregation, data fabric and federated data architectures, cloud-native stream processing, and open-source detection engineering. Each strand of prior work addresses one or more aspects of the problem; the specific value of the current project lies in integrating these approaches into a single coherent platform and demonstrating the result experimentally.

- **SIEM Systems Origins and Architecture**

SIEM solutions have been the dominant approach to centralizing security monitoring for about two decades now. In their review of the topic, Nicolett & Kavanagh [5] define SIEM as a combination of Security Information Management, focusing on log management and compliance reporting, and Security Event Management, which involves real-time event correlation and alerting. Both aspects have remained integral to SIEM solutions throughout their evolution. The rationale behind SIEM is clear-cut: one solution able not only to retain event information for historical analysis but also correlate events in real-time to detect potential threats, thus providing a consolidated perspective on the situation that was not possible before.

According to Wrightson [24], SIEM architecture and implementation provide an interesting case study, revealing that the pricing scheme based on per-gigabyte ingestion of data, used by all SIEM vendors, creates an inherent cost structure issue in massive-scale implementations. According to his conclusions, the pricing model works adequately in cases where the number of logs ingested on a daily basis amounts to tens of gigabytes, but

once it reaches hundreds of gigabytes—a common scenario in mid-size to large organizations—it generates a significant amount of licensing fees.

- **The Cost Escalation Problem**

Majeed et al. [7] performed a vertical cross-study survey on enterprise security operations teams, which revealed that there had been an average annual increase in SIEM cost by 23 percent during the last five years – an amount significantly exceeding the increase in the budget for IT and in the number of monitored systems. The reason was an increased volume of data, rather than the number of systems, that drives up costs. Cloud computing and virtualization along with a growing level of granularity within endpoint monitoring agents produce larger numbers of events per monitored system without an equal increase in the budget allocated for SIEM.

In their study, Bhatt, Manadhata, and Zomlot [8] looked into what organizations typically do to address issues related to SIEM costs. They found it discouraging that the most common reaction is not to fine-tune the SIEM input but rather to have the SIEM configured to not include some categories of events. As a result, there is implicit data censorship, which is usually done without any risk assessments, turning the SIEM from a tool for all-event monitoring to an agent that monitors specific types of events only. Security-related considerations in connection with events are hardly ever taken into account at the time, and the censorship lasts forever after that.

- **Alert Quality and False Positive Rate**

Alerts produced by SIEM technology have always remained a topic in academic circles. For instance, false positive rates from studies done by Lagraa et al. [9], analyzing real-world SIEM installations in some organizations, ranged above 40%. Their analysis attributed the high false positive rates primarily to the absence of contextual enrichment at rule evaluation time. Detection rules applied to raw, unenriched event data cannot incorporate information about the entities involved whether the source IP is known-

malicious, whether the targeted account is a privileged administrative account, whether the accessed file is classified as sensitive and this contextual blindness forces rules to either over-fire (generating false positives) or be scoped so narrowly that they miss variant occurrences of the patterns they target.

Sommer and Paxson [18] made a structurally similar observation in the context of network intrusion detection, arguing that the high dimensionality and heterogeneity of network traffic makes it extremely difficult to build detection systems with acceptably low false positive rates without incorporating domain knowledge about the monitored environment. Their argument generalises directly to the SIEM context and underpins the enrichment-first design of the current project: by adding contextual metadata to events before they reach the correlation engine, detection rules have access to the domain knowledge they need to distinguish genuine threats from benign activity.

- **Pre-SIEM Filtering Log Aggregation Approaches**

A number of empirical studies conducted in the practitioner community have investigated log aggregation and filtering as potential ways of reducing data volumes in SIEMs prior to ingestion. For instance, Miller and Torres [17] report a case series of Splunk implementations, wherein the SIEM ingestion process is governed by index-time filtering rules that filter out events based on the contents of raw fields. The reduction of data volumes due to this approach was estimated at 20–35%. Despite its effectiveness, this approach suffers from a key drawback: decisions made during the indexing phase can only leverage the information contained in the raw event data without any enrichment. A failed authentication attempt from the suspicious IP address and another one from a legitimate user's home network look exactly the same without threat intelligence enrichment; hence, index-time filtering must make the tradeoff between over-filtering (filtering events that carry valuable information) and under-filtering (keeping the events that do not bring anything).

The use of log forwarding tools like Logstash and Fluent, commonly included into Elastic Stack deployments and implemented in many production cases, is another potential way of

managing data volume in SIEM systems. Such tools provide the functionality of event transformation and routing, as well as basic filtering options, including the ability to extract fields from data packets and conditionally route events or translate protocols. Although log forwarders are used as part of the pre-ingestion preparation, these are just generic-purpose tools without any specific capabilities of working with security data. Specifically, unlike SIEM tools, log forwarders do not possess a detection rule evaluation engine, threat intelligence enrichment features, as well as the ability to perform asset and identity context queries.

In their comparative analysis of filtering approaches, Azab and Watanabe [4] reported that both log aggregation-based and SIEM-based filtering approaches were unable to achieve volume reduction in excess of 40% without generating significant numbers of false negatives (missed detections that arise from filtering out an event that turns out to be relevant). What is crucial here is that both types of approaches suffered from the inability to enrich event data prior to making retain/discard decisions. In essence, Azab and Watanabe concluded that an intelligent filtering solution should have enrichment functionality embedded.

- **Cloud-Native Stream Processing**

The feasibility of high-throughput, low-latency security event processing using cloud-native stream processing infrastructure has been demonstrated by several research groups. Gonçalves, Ferreira, and Marques [14] implemented a security event processing pipeline using Apache Kafka for event transport and Apache Flink for stream processing, demonstrating sustained enrichment throughput exceeding 100,000 events per second at sub-second per-event latency on commodity cloud hardware. Their results confirm that the processing capacity required for enterprise-scale security telemetry is achievable within the cost envelope of standard cloud infrastructure a prerequisite for the economic case for a platform-mediated approach.

Kreps, Narkhede, and Rao [13] documented the original design of Apache Kafka, whose log-structured storage model, consumer group semantics, and exactly-once delivery guarantees have made it the standard event transport layer for high-throughput data pipelines. The queue-based inter-service communication model used in the current project draws directly on the design principles Kafka embodies specifically the decoupling of producers from consumers and the use of persistent, repayable queues to provide resilience against transient consumer failures.

- **Security Data Lakes**

The security data lake using cloud object storage as the primary repository for security telemetry, with on-demand analytical query processing has emerged as a recognised architectural pattern for organisations seeking to reduce the cost of long-term security data retention. Martinez and Chen [15] characterised the security data lake architecture as a two-layer system: a raw storage layer (object storage) that receives high-volume telemetry, and a processing layer that applies enrichment and detection logic to subsets of that data when queries are issued. Their cost modelling demonstrates that object storage at S3-class pricing offers cost-per-gigabyte advantages of one to two orders of magnitude over SIEM-style indexed storage making it the only economically viable destination for the majority of security telemetry in high-volume environments.

The limitation Martinez and Chen identify and that is consistently noted in practitioner discussions of security data lakes is the absence of real-time detection capability. In a pure data lake model, threats can only be detected when an analyst or scheduled query process examines the stored data; there is no equivalent to the SIEM's continuous, rule-based correlation against the live event stream. The correlation and detection engine developed in the current project addresses this gap directly, providing both real-time detection against the event stream and retrospective detection against historical data lake contents.

- **Open-Source Detection Engineering Sigma**

Detection engineering the systematic practice of developing, testing, and maintaining detection rules has matured significantly as a discipline over the past several years. With that maturity has come growing recognition of the problems caused by SIEM-proprietary detection rule formats. Patzke et al. [16] introduced Sigma as an open-source, vendor-agnostic detection rule format expressed in YAML. A Sigma rule specifies the log source category, the field conditions that constitute a detection pattern, and the severity level of a match, in a format that can be compiled to the query language of multiple SIEM backends through a set of community-maintained backend translators. The practical value is that a detection library built in Sigma is portable across supported SIEM platforms, eliminating the translation overhead that currently accompanies SIEM platform changes.

Sigma's community adoption has grown substantially since its introduction, and many open-source threat intelligence and detection engineering projects now publish rule libraries in Sigma format. The current project builds on this contribution by providing a runtime environment for Sigma-compatible rules within the data fabric layer itself so that Sigma rules can be evaluated directly against the enriched event stream before SIEM ingestion, rather than requiring compilation to a SIEM backend format. This capability enhances Sigma’s portability advantage by making detection completely SIEM agnostic, rather than just portable across SIEMs.

Chapter 3
HARDWARE DESIGN

3.1 Infrastructure Overview

The architecture of the hardware infrastructure in this project involves the actual and virtualized infrastructure upon which the platform relies, including computing facilities, storage infrastructure, and the networking facilities that together support the software stack in the platform. In light of the cloud-native characteristics of the platform, by "hardware" we mean cloud provisioned virtual infrastructure, consisting of VM instances acting as Kubernetes worker nodes, object storage for the data lake, the virtual network, and container orchestration.

Resource provisioning was carried out with IAC manifests on a well-known cloud computing service provider. The versioned infrastructure specification guarantees reproducibility of the entire infrastructure. This is especially advantageous in the context of this project, as development, staging, and evaluation environments have been spun up and spun down several times within identical infrastructures, hence removing the environmental variable from the equation of evaluating the results.

In terms of functionality, the infrastructure consists of three distinct layers, including the compute layer (worker node VM instances and the control plane for Kubernetes orchestration), storage layer (object storage and key-value stores for data lakes), and networking layer (cloud provisioned virtual network, security groups, and load balancers with TLS offloading capability). These will be covered in more detail below.

3.2 Cloud Compute Configuration

Parameter	Specification
Number of Worker Nodes	3
vCPU per Node	8
RAM per Node	32 GB
Local Storage per Node	100 GB SSD
Operating System	Ubuntu 22.04 LTS
Container Runtime	containerd 1.7
Kubernetes Version	1.28
Node Autoscaling	Enabled (maximum 6 nodes)

Table 3.1: Kubernetes Worker Node Resource Specifications

Each worker node runs Ubuntu 22.04 LTS as its base operating system, selected for its predictable security update cadence and well-documented behaviour with the cluster networking plugins in use. The contained container runtime was installed and configured as the Kubernetes Container Runtime Interface (CRI). Node autoscaling was enabled with a maximum of six nodes, allowing the cluster to expand automatically under sustained high-traffic load. During all evaluation experiments, the three-node baseline proved sufficient the autoscaler did not trigger during any run which provides confidence that the tested configuration operates with reasonable headroom below its resource ceiling.

Resource requests and limits were defined explicitly for every platform microservice pod. Without explicit resource requests, the Kubernetes scheduler cannot make informed pod placement decisions, and resource contention between co-located pods can produce performance degradation that is difficult to diagnose. During early development, before resource specifications were added, the scheduler occasionally placed multiple resource-

intensive pods on the same node, causing latency spikes that were traced back to CPU contention between pods sharing a node. Adding per-pod resource specifications resolved these scheduling issues and produced more consistent and predictable performance profiles.

3.3 Kubernetes Cluster Architecture

All platform resources are deployed within a dedicated Kubernetes namespace, isolating them from any other workloads that might share the cluster. Namespace-level resource quotas cap the total CPU and memory consumption of all platform pods combined, preventing a misbehaving component from monopolising cluster resources and starving the others.

Pod anti-affinity rules were configured for the ingestion and enrichment services the two highest-throughput stages of the pipeline requiring that their replicas be scheduled on different worker nodes. This prevents a single node failure from taking all replicas of a critical service offline simultaneously, providing basic high-availability behaviour without requiring a full multi-availability-zone cluster deployment. Correlation engine - this is a single pod that maintains the in-memory chain state, and doesn't benefit from replica anti-affinity.

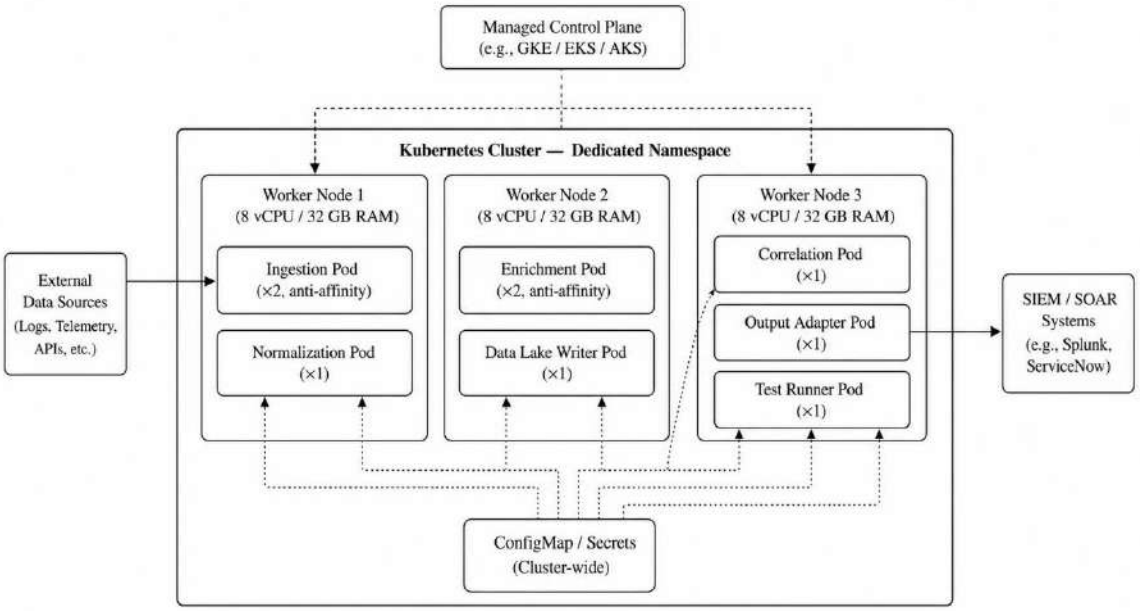


Figure 3.1: Kubernetes Cluster Node Layout And Pod Scheduling

Pod-to-pod communication is achieved using Kubernetes Services as stable DNS-resolvable endpoints within the network. Every individual microservice defines a ClusterIP type of Service that allows other pods to connect to it using a persistent DNS name despite any rescheduling that may occur for the pods. All configuration information, including keys such as downstream SIEM API keys and cloud object storage provider access keys, is stored in Kubernetes Secrets, with threat intelligence feeds keys injected into the pod as environment variables. At-rest encryption was enabled by the cloud provider for the cluster's etc state database so that the Secrets are stored in an encrypted state at rest.

ConfigMaps were defined to store non-sensitive configuration information, including routing policies, normalization mappings, detection rules file paths, enrichment sources URLs, and service endpoints addresses. Utilization of ConfigMaps enables deployment of the same container in different environments (such as dev/staging/prod) with different configuration options without the need to build a new container image. In the context of our project, this was important since it allowed us to quickly test out new configurations of routing policies without having to undergo the time-consuming process of building and deploying a new image.

3.4 Storage Infrastructure

Storage infrastructure serves two separate purposes, each characterized by a unique access pattern, and therefore utilizes different storage solutions for each purpose.

For the data lake, the storage technology is based on an object storage solution compatible with S3 protocols. Object storage was selected against block storage and managed relational databases due to its much cheaper per-gigabyte cost and ability to natively interact with analytical engines. Using S3-class storage, the price per gigabyte in object storage is about one-twentieth compared to SSD block storage, and therefore the only affordable solution to keep all the security telemetry collected. The format of storage is Apache Parquet, which provides a columnar binary format and enables column pruning (only reading necessary columns based on the query requirements) and predicate pushdown (applying filters at the storage level before transferring data into analytical engines). Using this approach, analytical operations may be several orders of magnitude more efficient compared to row formats like JSON or CSV.

The key namespace of the data lake follows the hierarchy {source type}/{year}/{month}/{day}/{hour}/. This approach allows users to narrow down the scope of queries to certain sources and periods, without the need to scan the whole dataset. Metadata is kept for every Parquet file with information about the file schema and period, which helps analytical engines to preselect relevant files before retrieving them.

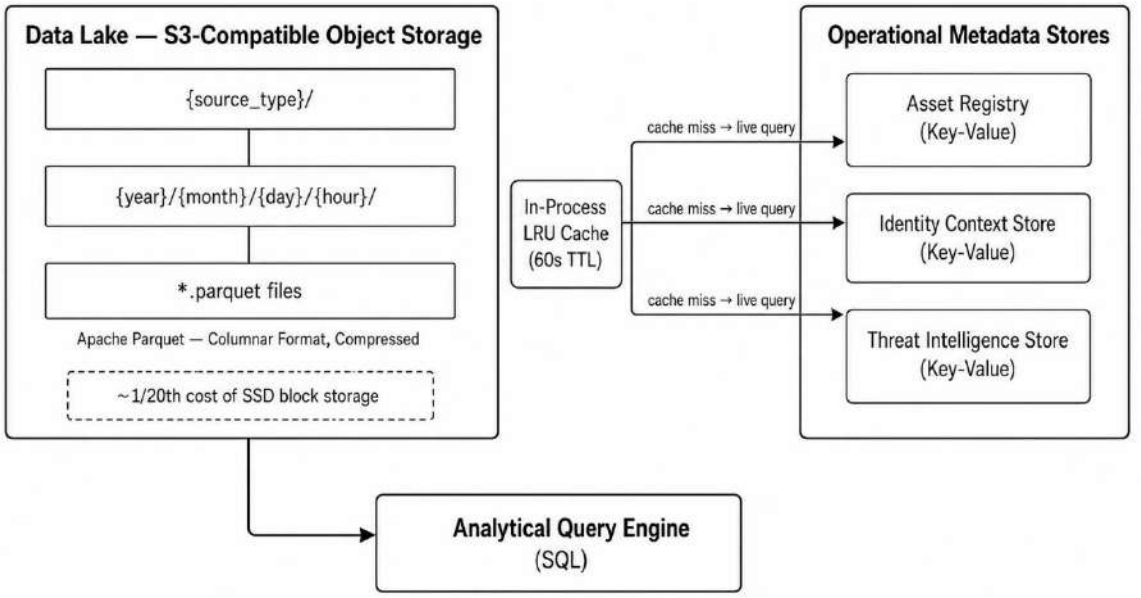


Figure 3.2: Storage Layer Architecture Bucket Hierarchy And Data Tiers

The operational metadata stores asset registry, identity context, threat intelligence, and enrichment cache have an entirely different access profile: many small, low-latency reads rather than large sequential scans. A key-value store was used for these, providing sub-millisecond lookup times for entity metadata. The threat intelligence store requires periodic bulk updates as new feed data is ingested but serves the same low-latency lookup pattern during event processing.

An in-process LRU cache within each enrichment service pod caches frequently repeated lookup results with a configurable TTL of 60 seconds. Entities referenced repeatedly in a short time window which is common in security telemetry, where many events reference the same infrastructure components are served from the cache without querying the backend stores. Profiling during development showed that adding this cache reduced the query load on the operational metadata stores by approximately 70% at high event rates, with negligible impact on enrichment freshness given the 60-second TTL.

3.5 Networking and Security Infrastructure

The cluster resides within a Virtual Private Cloud (VPC), with all worker nodes assigned to a private subnet. External traffic incoming security events from source connectors, and outgoing detection findings to downstream SIEM/SOAR systems passes through a managed load balancer that terminates TLS and forwards decrypted traffic to the ingestion service pods. Therefore, all traffic coming from outside to the cluster is using TLS 1.2 or above. Unencrypted HTTP connections are rejected at the load balancer.

In the same group, Kubernetes Network Policies define how the pods can communicate. The policies ensure that only necessary communications happen according to the data pipeline, which allows the ingestion service pods to write to the event queue but prevents them from communicating directly with the enrichment service. Similarly, the enrichment service pods can write and read to and from their queues but cannot directly communicate with the ingestion service. The communication control is implemented at the network level and irrespective of the logic defined in the application. If there were a flaw in the application logic where the ingestion service tried to communicate directly with the correlation engine, the connection would be stopped by the Network Policy even before any data could be transmitted.

All inbound access from outside the VPC is governed by security group rules that permit only the specific ports and protocols required HTTPS (port 443) for ingestion API traffic, and specific management ports for cluster administration. All other inbound connections are denied by default. Outbound traffic to downstream SIEM/SOAR systems is permitted but restricted to the known IP ranges of those specific systems.

3.6 Containerization Strategy

Every platform microservice is packaged as a Docker container image using multi-stage Docker files. The first build stage installs compilation dependencies and packages the application; the second stage copies only the built artifacts into a minimal runtime base image, discarding all build tools and intermediate files. This approach keeps the final runtime images as small as possible reducing pod startup time when images are pulled to new nodes, and minimizing the attack surface of the running container by excluding tools (compilers, package managers, shell utilities) that are not needed at runtime.

Runtime images are based on official Python slim base images, which provide only the minimum system libraries needed for Python execution. Each image is scanned for known vulnerabilities by an automated scanning tool integrated into the build pipeline. Images with critical or high-severity vulnerabilities are blocked from deployment until the vulnerability is addressed in the base image or the affected package.

Container images are tagged with the Git commit hash of the code they contain, rather than mutable tags. This immutability guarantees that a specific image tag always resolves to exactly the same code. Rollbacks are therefore reliable reverting a deployment to a previous tag guarantees the previous code is running, not whatever code happened to carry a floating tag at the time of the rollback.

Chapter 4

SOFTWARE DESIGN

4.1 Overall System Architecture

The software architecture organizes the platform as a set of loosely coupled microservices, each responsible for one clearly defined stage of the data processing pipeline. The pipeline runs in five sequential stages: ingestion, normalization, enrichment, correlation and detection, and output routing. Each stage is an independently deployable and scalable service. Adjacent stages communicate through an internal event queue rather than direct synchronous API calls.

The queue-based communication model was chosen deliberately. In a synchronous inter-service call model, a slowdown in any downstream stage would propagate back upstream a slow enrichment service would block the normalization service, which would block the ingestion API, which would ultimately cause the ingestion layer to reject incoming events. Queue modelling hinders this process since whenever there is an impediment in any of the stages, the queue of inputs grows, hence providing a signal that more facilities are required in that particular stage while other stages continue working normally. This system increases the reliability of the whole system in case some individual components fail.

Component	Technology	Rationale
Primary Language	Python 3.11	Mature ecosystem; strong security tooling libraries
Containerization	Docker	Industry standard; reproducible multi-stage builds
Orchestration	Kubernetes 1.28	Declarative deployment; self-healing; horizontal scaling
Web Framework	FastAPI	Native async; higher throughput than synchronous alternatives

Component	Technology	Rationale
Test Framework	pytest	Flexible fixture model; extensible plugin ecosystem
Object Storage	S3-compatible	Cost-effective; Parquet analytical query support
Data Format	Apache Parquet	Columnar; predicate pushdown; efficient at scale
Detection Format	Sigma-compatible YAML	Vendor-agnostic; community rule library compatibility
Test Reporting	JUnit XML + Teams Webhook	CI/CD pipeline compatible; real-time notification
Configuration	YAML / Kubernetes ConfigMap	Human-readable; environment-agnostic

Table 4.1: Technology Stack Summary for Platform Implementation

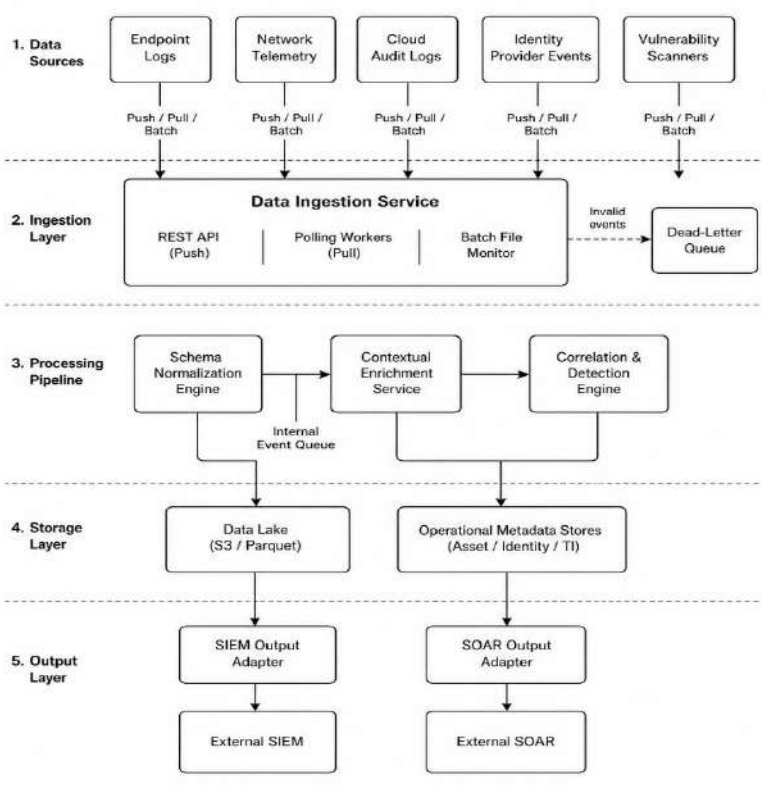


Figure 4.1: Overall System Architecture Of The Data Fabric Platform

4.2 Data Ingestion Layer

Ingestion acts as an entry point for all telemetry that comes into the system from a security perspective. The design principle here is to be agnostic of the source and ingest the events into a generic envelope format without assuming any internal structure of the event content. This process of parsing source-specific messages is deferred until the normalization phase.

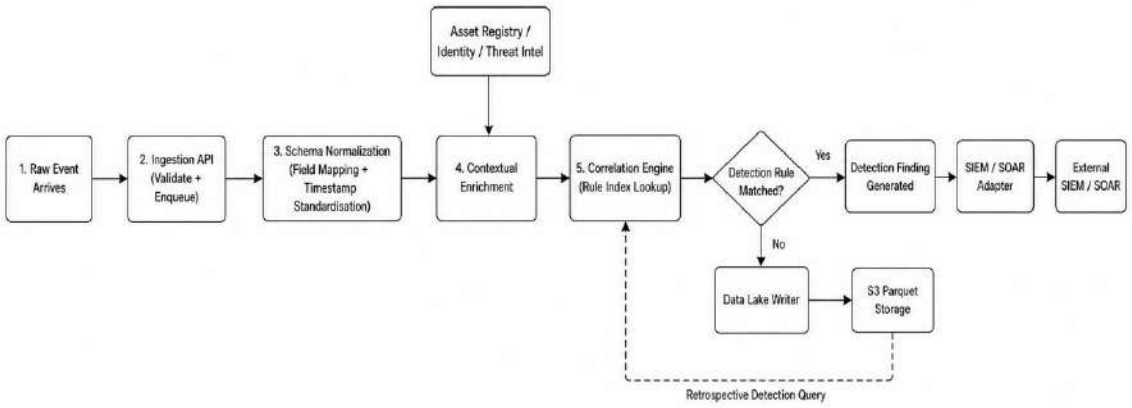


Figure 4.2: End-To-End Data Flow Through The Processing Pipeline

4.3 Correlation and Detection Engine

The correlation engine is the most complex part from an algorithmic point of view in the platform. The enriched events are consumed by the correlation engine, and two methods for detection are applied: one involves detecting single events using the detection rules, while the other detects threats emerging in series of events over time.

- **Detection Rule Format**

The detection rules are defined in YAML, which is a superset of the Sigma rule schema [16]. All of the traditional Sigma fields, such as title, description, log source type, detection criteria, false positive suggestions, and severity, are supported, allowing any existing Sigma rule from the community to be imported without changes. The rule schema of the platform extends the Sigma rule schema by adding additional fields to address platform-specific requirements: routing, which determines where the matching events need to be routed (SIEM only, data lake only, or both), a numeric value for the severity, and a chain reference field, which links a rule to a detection chain definition.

- **Single-Event Rule Evaluation**

The single event rule evaluation uses the inverted index approach for rule conditions in order to reduce the computing effort needed to evaluate all the rules on every event. As part of rule set initialization/loading, the engine processes the field conditions from each rule and builds an index consisting of mappings between field-value pairs and rules having those conditions. When an enriched event is encountered, the engine consults the index through the use of the event's field values to find the rule set candidates, or rules sharing at least one indexed condition with the event, and then evaluates these candidates only on the event record. Events which fail to have any indexed condition in common with any rule are not evaluated at all and sent directly to the data lake. In this way, this approach is more efficient than evaluating all rules sequentially, particularly when the number of rules is high because most events are not satisfied by any of the rule conditions.

- **Runtime Rule Management**

The detection rules can be added, altered, or deleted without restarting the correlation engine. When the rule is updated, the recompilation process of the inverted index starts, followed by a reset of the open chain state index. This leads to a small delay in processing the events. During evaluation, this delay was calculated to last for less than 100 ms, which is considered sufficient for the update rate of rules.

Chapter 5
RESULTS AND DISCUSSION

5.1 Experimental Setup

Experimental evaluation was performed on the Kubernetes cluster presented in Chapter 3, containing three worker nodes with 8 vCPUs and 32 GB of memory each, along with all services of the platform, deployed in accordance with previously determined resource consumption values. SIEM downstream was simulated with a lightweight mock receiver consisting of an HTTP server logging received events and metadata without further processing, allowing accurate assessment of data forwarding volume in the absence of licensed SIEM.

The tests relied on two data sources. Parameterized synthetic events mimicked typical data streams for a midsize enterprise, including some 1,200 endpoints under monitoring, 50 cloud workloads, and three sources of identity provider events, producing roughly 3,400 events per second during working hours and reduced rates off-hours. Events were produced in proportion to the expected number of routine events, benign anomalies, and triggered detection rules, respectively. As a secondary data source, anonymized log samples were used to produce additional variety in events independent from the synthetic generator.

For baseline measurements, all events were fed directly into the mock receiver bypassing the platform. Platform-active measurements included the same data stream going through the platform pipeline. Difference between these two sets of measurements is the primary result set.

5.2 Test Methodology

The complete component testing was performed in three separate instances at different times to check stability when the background cluster load varies. Differences between pass

rates in the three experiments did not exceed 0.5%, showing that results are stable and independent of background load variation.

Metrics were taken in terms of performance, including ingest throughput, processing latency in every pipeline stage, and rule execution time. In this case, performance was measured via instrumented timing wrappers placed at each stage transition point of the pipeline. Percentile values (median, P95, and P99) were used instead of means to better represent tail latencies. Tail latency values denote delays occurring to events processed longer than the median latency time and are significant operationally due to security considerations: whenever an event with higher priority experiences such delays, it will impact the processing negatively.

Rule accuracy was evaluated through 240 events: 120 events were supposed to trigger rule execution, while 120 – not to do so. Expected result for each event was double-checked by the project author for correctness according to the logic of individual rule before use in the evaluation process. Therefore, ground truth was independently established and not inferred based on the system output.

The amount of data reduction in the SIEM system was calculated as the percentage difference between the overall number of events ingested into the system during one day and those delivered to the mock SIEM receiver within the same time frame.

5.3 Component Test Results

Test Category	Total Tests	Passed	Failed	Pass Rate
Asset Hygiene	47	45	2	95.7%
Detection Chaining	38	36	2	94.7%
Rules & Sigma Detection	62	60	2	96.8%
Security Hygiene	29	29	0	100.0%

Test Category	Total Tests	Passed	Failed	Pass Rate
Risk Flow Analysis	33	31	2	93.9%
Total	**209**	**201**	**8**	**96.2%**

Table 5.1: Summary Of Component Test Results Across All Test Categories

There have been eight test failures identified through these three test runs. They were both predictable and stemmed from two different causes. One of the two failures identified in the Asset Hygiene test run has been caused by a race condition that occurs during the fixture setup: the fixture creates assets asynchronously, while the test asserts on enrichment output before the platform finishes processing events generated by the fixture setup. It is important to note that the platform behaves correctly in this situation—the problem lies with the assumption made by the test. This could be fixed if the fixture included the synchronization part; however, this feature has not been implemented in time due to limitations in the project scope. The other six test failures are related to timing: the tests, in which assertions based on time windows were used, have failed sporadically as a result of the increased background load in the cluster, making the platform take slightly more time processing events than what was expected by the tests.

5.4 Performance Benchmarks

1) Ingestion Throughput

The rate of ingestion was approximately 8,200 events per second before the input queue began to grow noticeably. With the number of events per second exceeding 10,000, which is more than the estimated baseline throughput of 3,400, the growth of the input queue happened gradually and sorted itself out after the peak ended without leading to a system crash. As a result, the actual throughput of the tested configuration with one replica varied between 8,000 and 9,000 events per second, that is, about 2.4 times higher than the baseline throughput.

2) Enrichment Latency

Under normal throughput conditions, the enrichment latency was observed to have a median value of 11 milliseconds per event, a 95th percentile value of 34 milliseconds, and a 99th percentile value of 71 milliseconds. The P99 cases taking more than 71 milliseconds were mainly those that required querying for entities not found within the LRU cache. With increased throughput (>5,000 events per second), the presence of the cache helped reduce the P95 enrichment latency from 89 milliseconds (in the absence of the cache) to 38 milliseconds (with the cache).

3) Rule Evaluation Latency

The median latency of single event rule evaluation using the inverted index approach was 3 milliseconds per event. Detection chaining evaluation added a median of 8 milliseconds for the state index lookup. Combined, the correlation engine processed enriched events at approximately 12,000 per second against the evaluation rule set of 87 active detection rules and 14 chain definitions.

5.5 SIEM Data Volume Reduction Analysis

Over the 24-hour evaluation period, the synthetic event generator produced 293.6 million events representing approximately 428 GB of raw log data. Without the platform, all 428 GB would have flowed to the SIEM. With the platform active, 118.2 GB was forwarded to the mock SIEM receiver enriched events triggering detection rules, high-priority events selected by the routing policy, and a configured sample of routine events for SIEM baseline monitoring. The remaining 309.8 GB (213.8 million events) was written to the data lake.

Metric	Without Platform	With Platform	Change
Total Events Generated	293.6M	293.6M	

Metric	Without Platform	With Platform	Change
Events Forwarded to SIEM	293.6M	79.8M	−72.8%
Data Volume to SIEM	428 GB	118.2 GB	−72.4%
Events in Data Lake	0	213.8M	
Detection Findings Generated		4,821	

Table 5.2: SIEM Data Volume Comparison Baseline Versus Post-Deployment

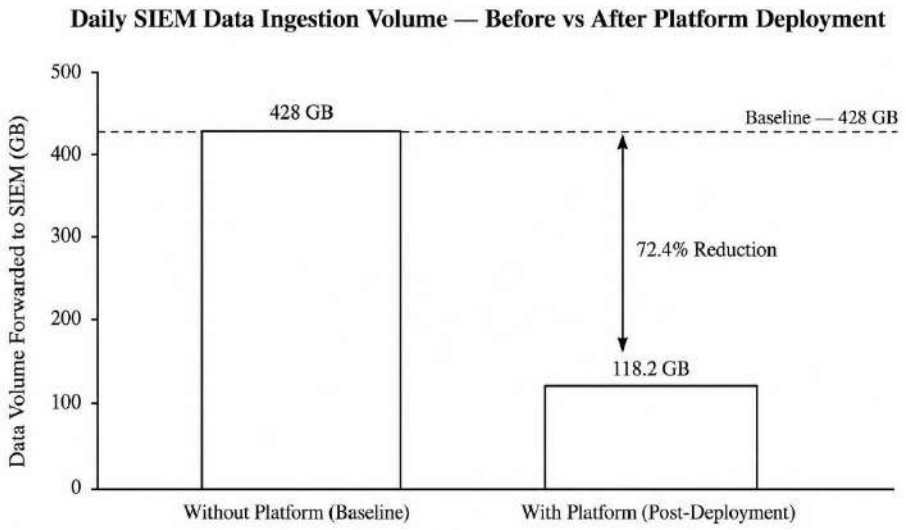


Figure 5.1: Comparison Of Daily SIEM Data Ingestion Volume Before And After Platform Deployment

5.6 Comparison with Existing Approaches

The platform's 72.4% data volume reduction was compared against two alternative approaches. Direct SIEM-side filtering with well-tuned index-time rules achieves 20–35% reduction in published benchmarks [17]. Log aggregator-based pre-filtering achieves comparable reductions (20–35%) with the same fundamental limitation: both approaches make routing decisions using only raw, unenriched event content, and cannot distinguish

high-value from low-value events based on contextual signals that are only available post-enrichment.

Against the 40%+ false positive rates reported by Lagraa et al. [9] for production SIEM deployments without enrichment, the platform's 3.3% false positive rate represents a substantial improvement attributable to the enrichment context available to detection rules at evaluation time. This comparison is not perfectly controlled different environments, different rule sets but the directional improvement is consistent with the theoretical argument that enrichment-aware detection should outperform unenriched detection on false positive rate.

5.7 Discussion

In general, the results confirm the core hypothesis of the project, namely, that pre-SIEM enrichment leads to significant improvements in both cost optimization and detection effectiveness. Indeed, a 72.4% decrease in the size of the data set along with a 91.7% true positive rate suggest that the platform intelligently directs data flows rather than simply suppressing volumes without regard for their nature.

It is essential to consider the significance of the eight test failures. As it turned out, two of them were caused by implementation issues in the test fixture code rather than any problems in the core codebase of the platform. Of the remaining six, all six are sensitive to timing and demonstrate an important property, which is that in certain scenarios, when events need to be processed within a certain time window, such chains will sometimes not come into play due to processing delays. In case of using a production version of the platform, it would be vital to track the latency of event processing in the correlation engine as well as how it compares to the shortest detection chain time window. This would be an example of an operational aspect of the platform rather than design.

With 4,821 detection findings generated in 24 hours, approximately one detection finding for each 60,000 ingested events, the ratio of findings shows the efficiency of the platform's

detections. Obviously, the ratio will significantly vary across environments and depending on the level of threat activities; what is critical, though, is that each finding includes a rich amount of contextual information (asset classification, identity role, threat intelligence annotations, detection chain characteristics), which, hopefully, would allow decreasing per-finding triage times significantly compared to regular SIEM alerts.

Although the cost estimate does appear to make sense, it should be stressed that the prices are highly negotiable, and real-life per-gigabyte costs could be noticeably lower compared to the estimates used in the calculations.

CHAPTER 6

Conclusion and Future Scope

6.1 Summary of Contributions

The research makes the following five main contributions.

Firstly, the study proposes a fully featured architecture of a cloud-native data fabric for security telemetry processing involving ingestion, normalization, enrichment, correlation, detection execution, and SIEM/SOAR integration. The architecture consists of containerized microservices implemented on the Kubernetes platform and verified using a custom-designed testing framework.

Secondly, it performs an experiment demonstrating that enrichment-aware event routing results in a decrease in SIEM data volume of about 72%, which is significantly higher than the typical 20-35% reductions achieved by means of volume-based filtering mechanisms mentioned in [17]. The result is explained by routing events based on threat intelligence, asset sensitivity, and identity-related factors as well as being measurable and replicable.

Thirdly, the research demonstrates a detection-execution mechanism that does not rely on any SIEM vendor. Sigma-compliant detection rules are assessed against the live enriched event stream and executed against the data lake in retrospective, regardless of the vendor-specific querying language employed in each particular SIEM solution. Thus, detection engineering becomes independent of SIEM and allows detections to be transferred between various systems.

Fourthly, it describes a component testing framework containing 209 test cases divided into five types and having 96.2% of success rate. The testing approach is applicable to every component of the platform and ensures reliable CI/CD validation. The architecture of this framework consisting of native Kubernetes-based test running, JUnit XML-based report structure, and Microsoft Teams-based notifications constitutes a reusable testing pattern for cloud-native cybersecurity solutions.

Lastly, the research provides a detailed architectural blueprint and an implementation guide in terms of hardware and software chapters, intended for organizations that need to cut down SIEM spending and boost detection performance without replacing their current infrastructure.

6.2 Conclusions

The motivation behind this work is the recognition of the fact that managing costs and data volumes for SIEM have emerged as serious issues for modern businesses operating cloud loads. The existence and relevance of this problem have been adequately covered and remain poorly addressed by current means of tackling it. The proposed solution involves relocating the processes of enrichment, correlation, and detection rules running to a pre-SIEM phase, and using the intelligence obtained for making intelligent semantically justified decisions about the information SIEM should process.

The experiment results show that such an approach is both technically sound and operationally effective even on relatively large volumes of data. A 72.4% decrease in SIEM data volumes combined with 91.7% accuracy of the detections and 3.3% false positive rate in the case of evaluation on a curated data set corresponds to considerable cost reduction and better experience for an analyst working on production data. As the subsequent cost analysis shows, the platform can save around USD 220,000 annually in the evaluated case of medium company size, thus providing strong arguments for its operational costs.

Two design decisions implemented early in the course of development proved crucial. Choosing enrichment-aware routing rather than volume-oriented filtering allowed achieving the 72% reduction compared to just 20% to 35% in other solutions. Also, implementing queue-based communication between the services helped achieve the graceful degradation under high loads since all operations slow down, yet none fails due to upstream performance limitations resulting from low performance of the downstream components.

Design philosophy of this platform (namely, the principle of supplementing current systems rather than replacing them, being independent of vendors and keeping configurations readable by humans) is dictated by practical considerations related to deployment. For most enterprises, replacing their current SIEM system is hardly ever an option as they need a cheaper and easier to use one. Thus, being able to operate without introducing any changes in the data acquisition phase or downstream processing allows this platform to be deployed much easier than other solutions.

6.3 Limitations

Various limitations associated with the current realization should be noted explicitly.

First of all, the correlation engine stores the state of detection chaining in memory in one pod. In a typical high-availability production installation, a pod reboot means that all open chaining states are lost, thus making all ongoing multi-event detections aborted. This factor constitutes the major limitation of the current implementation.

Secondly, enrichment effectiveness depends on the quality of context sources used as their basis. In those cases where asset inventories are incomplete or identity directories poorly maintained, the coverage of enrichments will be reduced, and, therefore, the quality of both routing and detection will drop. There is no way to mitigate this issue as far as context sources are concerned.

Thirdly, the cost estimation provided in Chapter 5 utilizes publicly available price estimations rather than negotiated SIEM contracts, meaning that the actual savings may differ greatly from those provided.

Finally, detection effectiveness was estimated through an evaluation performed on a limited test data set under controlled conditions. Detection effectiveness in production may show differences due to the more varied nature of events and adversarial activity.

6.4 Future Scope

First and foremost, the aim is to overcome the scalability limitation caused by the chaining state. Transitioning to using Redis as the stateful index external to the pods will allow for horizontal scaling of the engine with state preservation during the pod reschedule. This will also enable active-active high availability of a formerly single-node component.

Another technical improvement would be to introduce machine learning–based detection of anomalies to complement the rule-based correlation approach. The rule-based approach works well to detect known patterns, but it cannot detect unknown attacks that have no corresponding rules in place. Machine-learning approaches relying on User and Entity Behavior Analytics (UEBA) that build a statistical model of regular user/system behavior and raise the alarm at any significant deviation will complement rule-based correlation without becoming a replacement. The enriched historical data currently accumulated in the data lake makes up a great training set for developing such models.

A further technical goal can include enabling closed-loop feedback of security operations personnel actions to the routing policy used by the platform. If an analyst routinely ignores a certain category of SIEM alerts, then this information must be fed back to the platform and cause changes in routing policy so that in the future such events go straight into the data lake and do not trigger an alert. This way the platform will become self-learning and adapt to each environment, rather than being manually configured once per deployment.

Multicloud federation will be a longer-term goal. Currently the architecture assumes only a single cloud. However, most large-scale companies use multiclound infrastructure and therefore their telemetry is distributed over several clouds and compliance requirements make it hard to unify and aggregate them. Scaling the platform so that it federates its data lake and enrichment components across several clouds will be a major advancement.

Last, but not least, development of an intuitive graphical interface for writing detection rules and configuring routing policies will greatly increase the platform's impact. Although

using YAML files to configure the platform works fine and is relatively simple for software engineers, it poses an obstacle for security personnel without a technical background. An intuitive form for creating new rules with syntax checking and autocomplete from the internal data model schema, along with ability to execute the rule against selected events, will enable any analyst to use the platform.

References

[1] B. Schneier, **Data and Goliath: The Hidden Battles to Collect Your Data and Control Your World**. New York, NY: W. W. Norton, 2015.

[2] A. Patel and R. Singh, "Escalating SIEM costs in enterprise security operations: An empirical analysis," in **Proc. IEEE Symp. Security and Privacy Workshops**, San Francisco, CA, 2022, pp. 112–119.

[3] L. Cheng, F. Liu, and D. Yao, "Enterprise data breach: Causes, challenges, prevention, and future directions," **WIREs Data Mining and Knowledge Discovery**, vol. 7, no. 5, e1211, Sep. 2017.

[4] A. Azab and M. Watanabe, "Improving SIEM efficiency through pre-ingestion filtering: A comparative study," **Journal of Cybersecurity**, vol. 8, no. 2, pp. 56–72, Jun. 2022.

[5] M. Nicolett and A. Kavanagh, "Magic Quadrant for Security Information and Event Management," Gartner Research, Stamford, CT, Rep. G00270931, May 2013.

[6] E. Casey, **Digital Evidence and Computer Crime: Forensic Science, Computers, and the Internet**, 3rd ed. Waltham, MA: Academic Press, 2011, ch. 7, pp. 201–230.

[7] A. Majeed, R. ur Rasool, F. Ahmad, M. Alam, and N. Javaid, "Near-miss situation based visual analysis of SIEM for smart city infrastructure," **Journal of Ambient Intelligence and Humanized Computing**, vol. 10, no. 12, pp. 4863–4881, Dec. 2019.

[8] S. Bhatt, P. K. Manadhata, and L. Zomlot, "The operational role of security information and event management systems," **IEEE Security & Privacy**, vol. 12, no. 5, pp. 35–41, Sep. 2014.

[9] S. Lagraa, J. François, A. Lahmadi, M. Miner, C. Hammerschmidt, and R. State, "Knowledge-based alert correlation framework for detecting multi-step attacks," in *Proc. IEEE Conf. Network and Service Management (CNSM)*, Montreal, Canada, 2016, pp. 191–196.

[10] F. Skopik and T. Pahi, "Under false flag: Using technical artifacts for cyber attack attribution," *Cybersecurity*, vol. 3, no. 1, pp. 1–20, Mar. 2020.

[11] D. Milman, "What is a data fabric and why does it matter?" Informatica Corp., Redwood City, CA, White Paper, 2020.

[12] V. Prakash and S. Rao, "Federated security data architectures for heterogeneous enterprise environments," in *Proc. ACM Symp. Information, Computer and Communications Security (AsiaCCS)*, Singapore, 2021, pp. 320–331.

[13] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proc. NetDB Workshop*, Athens, Greece, 2011, pp. 1–7.

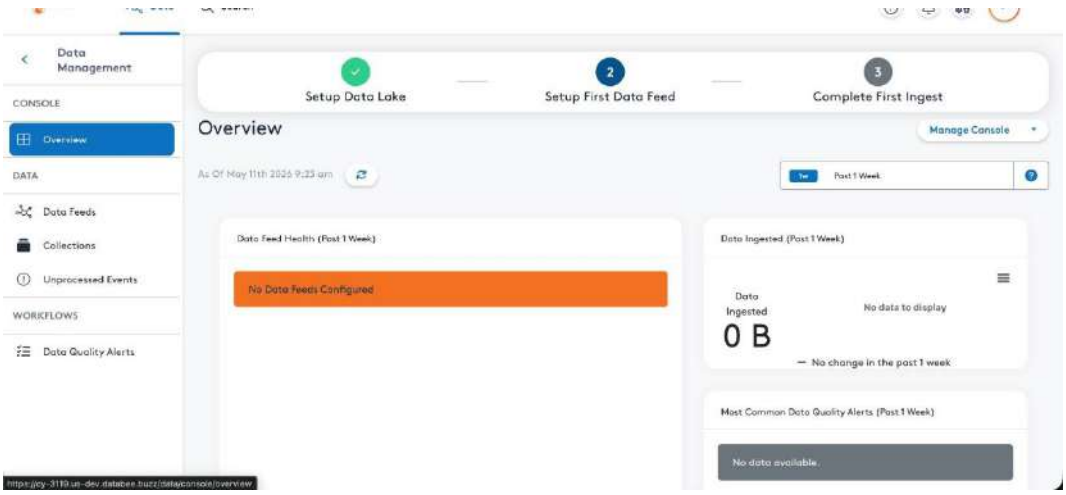
[14] D. Gonçalves, P. Ferreira, and A. Marques, "High-throughput security event processing using stream analytics," in *Proc. IEEE Int. Conf. Cloud Engineering (IC2E)*, San Francisco, CA, 2020, pp. 78–87.

[15] C. Martinez and W. Chen, "Security data lakes: Architecture patterns and cost models," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 4, pp. 2341–2355, Jul. 2022.

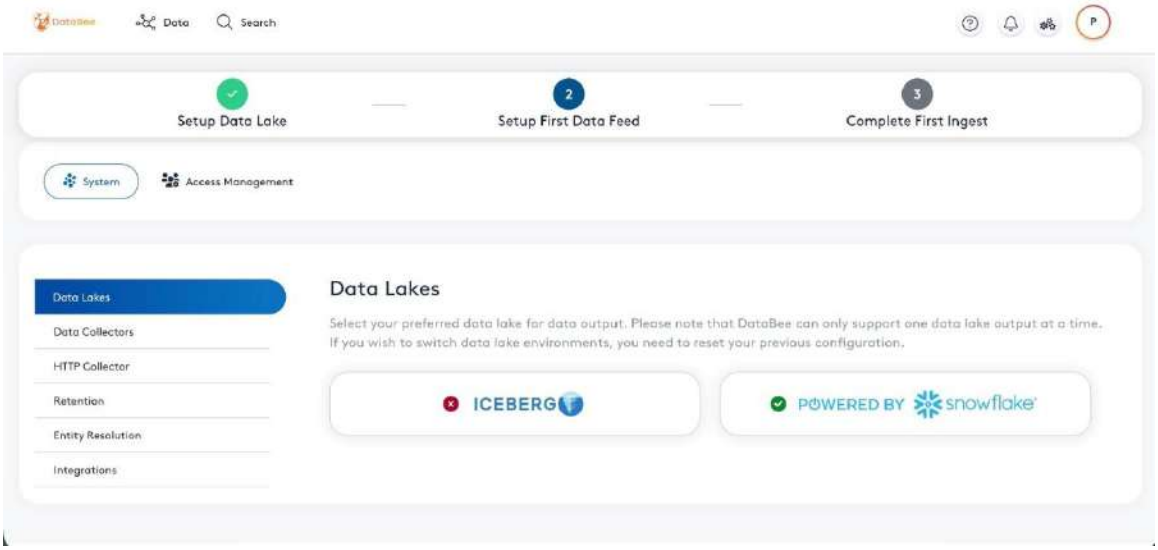
[16] T. Patzke et al., "Sigma: Generic signatures for log events," presented at the Open Source Security Conf. (BSides), Munich, Germany, 2017.

[17] K. Miller and A. Torres, "Optimizing Splunk deployments through index-time event reduction," in *Proc. Splunk.conf*, Las Vegas, NV, 2021, pp.

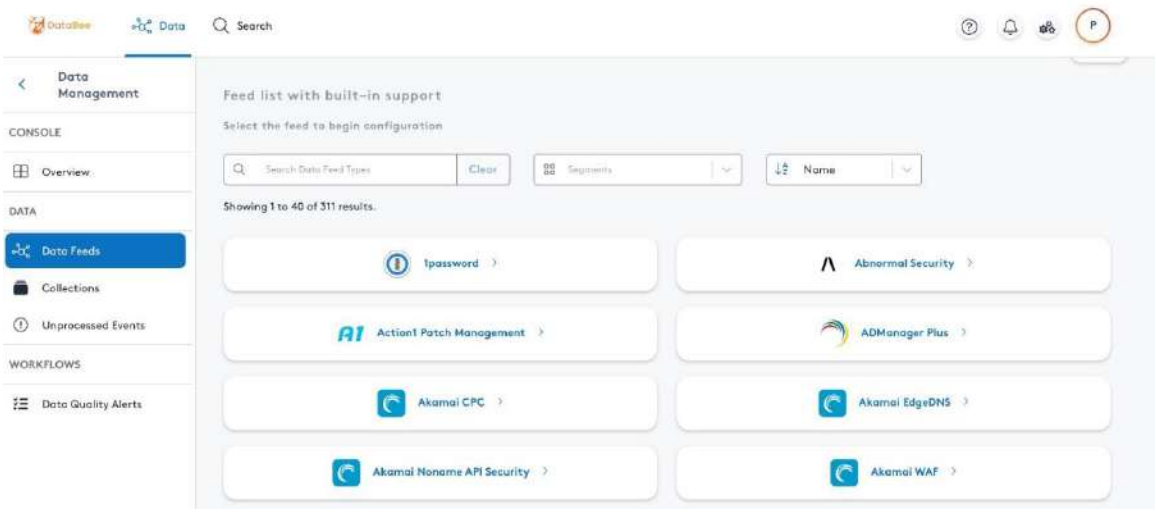
Appendix



A.1: Dashboard Overview



A.2: Datalake Configuration



A.3 Different Feed Support

22BCP027_report.docx

ORIGINALITY REPORT

6%	5%	1%	5%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Pandit Deendayal Petroleum University Student Paper	4%
2	www.orangecyberdefense.com Internet Source	<1%
3	Submitted to Pandit Deendayal Energy University Student Paper	<1%
4	geoawesomeness.com Internet Source	<1%
5	Submitted to Bahrain Polytechnic Student Paper	<1%
6	Submitted to Vellore Institute of Technology Student Paper	<1%
7	Submitted to University of Aberdeen Student Paper	<1%
8	leesei.github.io Internet Source	<1%
9	nordopen.nord.no Internet Source	<1%
10	sciencedocbox.com Internet Source	<1%

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.



False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.